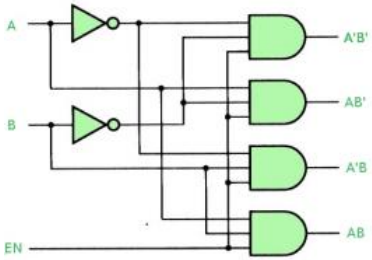
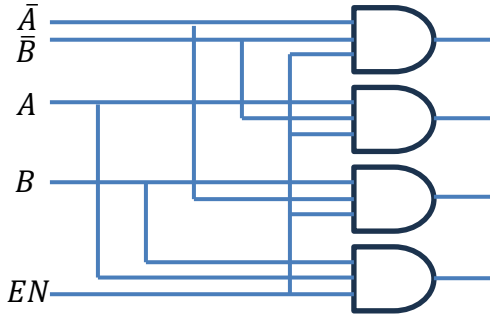
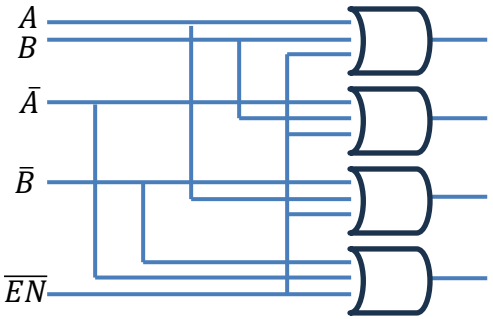
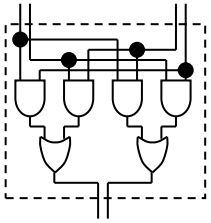
Home Exercise: Self Checking Circuits

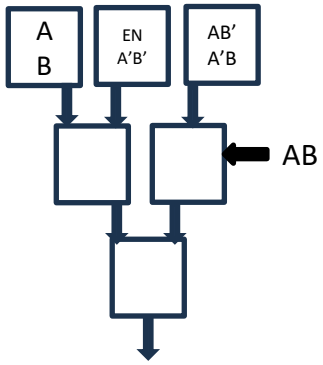
- *Design a Self-Checking 2:4 Demultiplexer with Enable*











Fault Masking

2110473 Fault Tolerant Computing

Prof.Behrooz Parhami, Graduate course materials, University of California, Santa Barbara
Daniel J.Sorin, M&C Publishers, Fault Tolerant Computer Architecture, Duke University, 2009
and others

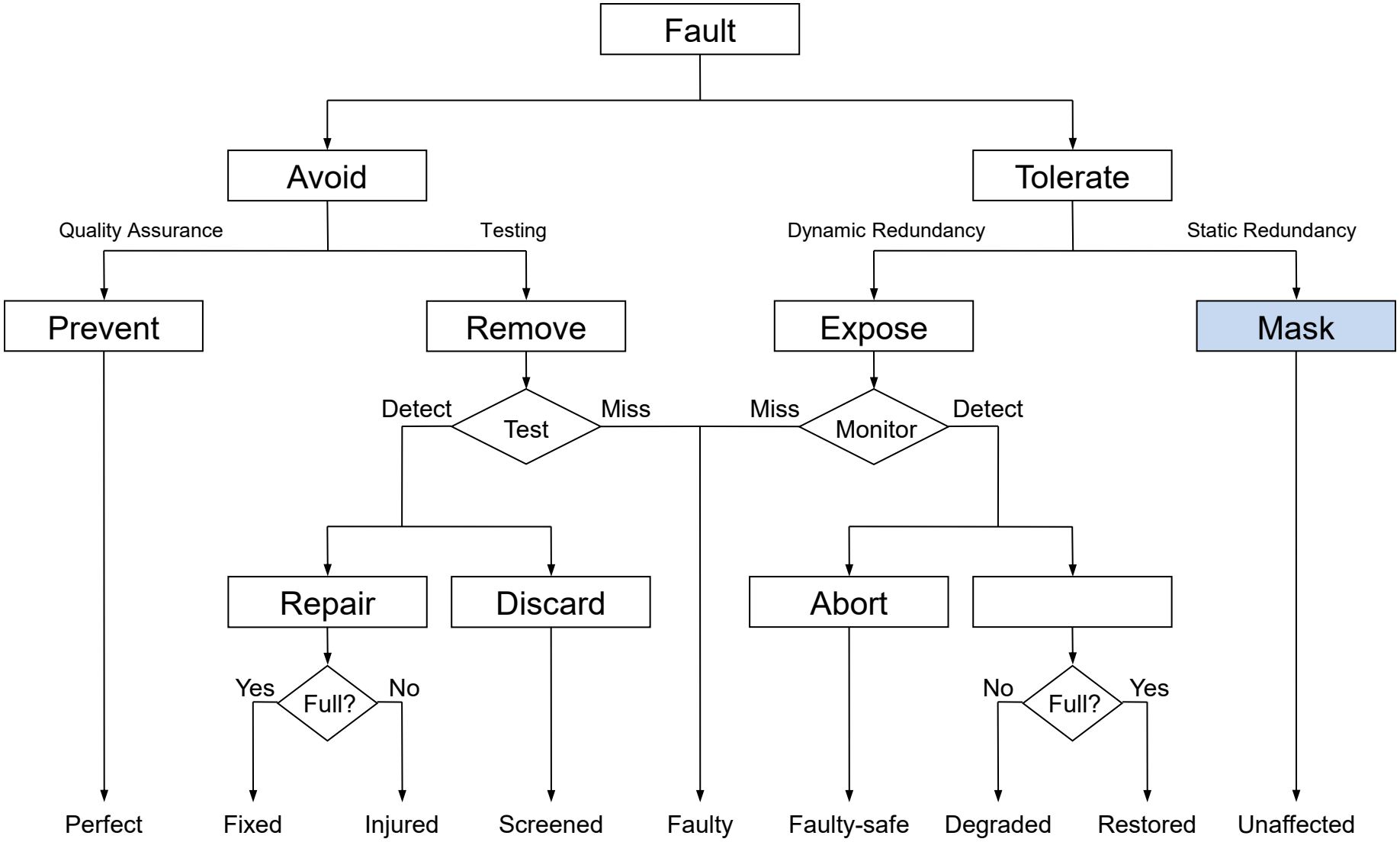
Schedule

- Mon 15 Sep, Self Checking✓
- Mon 22 Sep, No Class (Midterm)
- Mon 29 Sep, Fault Masking✓
- Mon 06 Oct, Midterm (Fault Tolerant Computing)
  - 13.00-16.00++/ #Eng4 no.18-15 (ON-SITE)
  - Open any Books including notebooks!!
- Mon 13 Oct, No Class (Comp.Holiday)
- Mon 20 Oct, Testing Digital Circuits (Online)

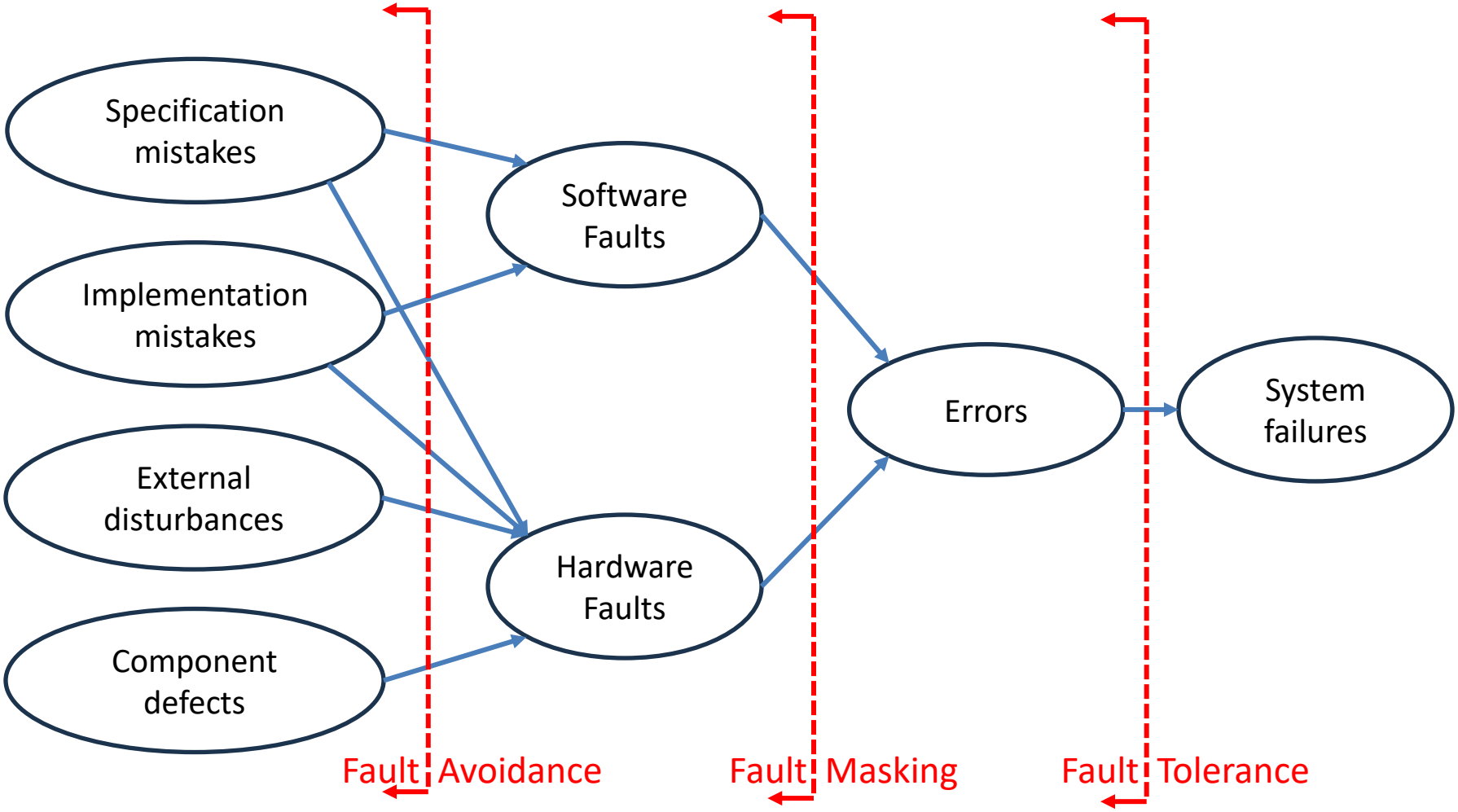
Contents

- Handling Faults
- Interwoven Redundant Logic
- Combinatorial Modeling
- TMR & its application
- Replication & Voting
- Alternating Logic
- Fail-safe Systems

Handling Faults

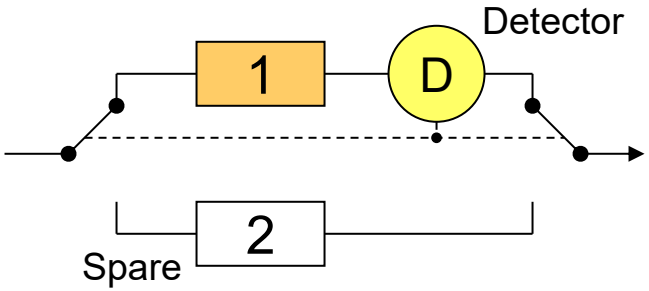


Design Techniques

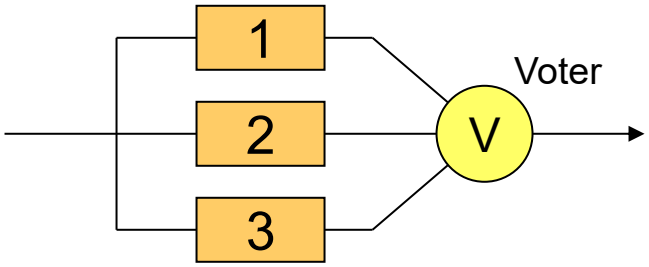


Some Options for Fault Tolerance

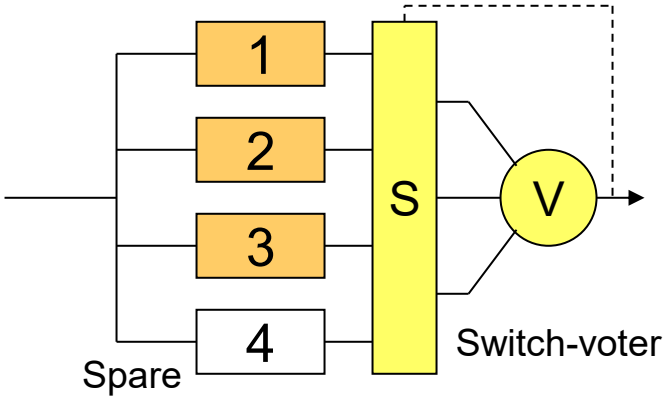
1. **Detect and replace**
   Dynamic redundancy (cold/hot standby)
   Detection via
   -- coding, watchdog timer, self-checking
   -- duplication (pair-and-spares)



2. **Mask**
   Static redundancy
   May revert to simplex instead of duplex
   Design challenges include
   -- synchronization for voting
   -- voting on imprecise results

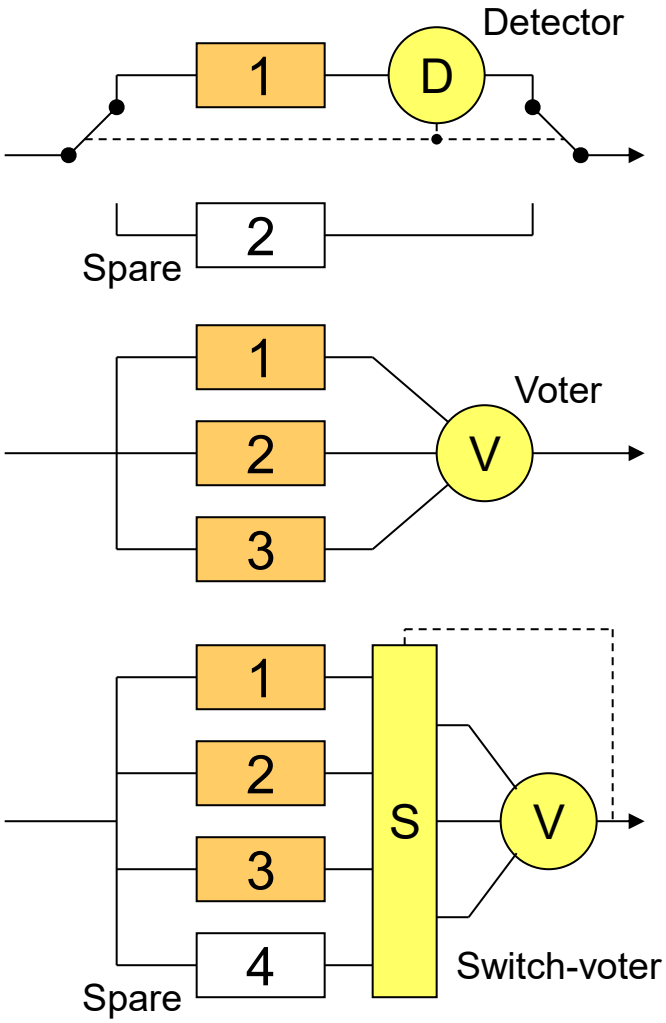


3. **Mask, diagnose, and reconfigure**
   Hybrid redundancy
   Fault masked at output, but diagnosed
   -- e.g., via comparison with voter output
   Faulty circuit is replaced by spare
   Becomes static upon spare exhaustion



Comparing Fault Tolerance Schemes

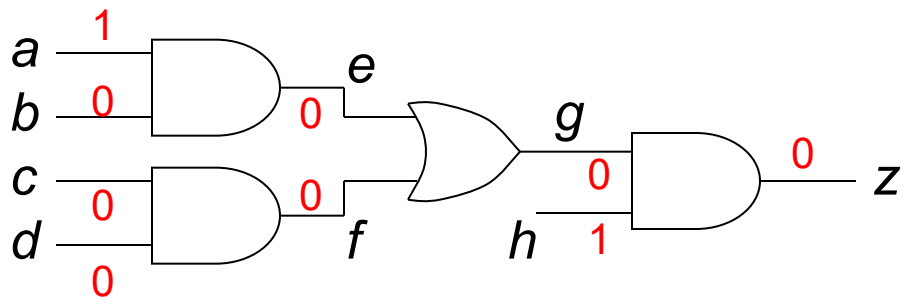
Advantages	Drawbacks
Less power (cold standby) Long life (just add spares)	Coverage factor Tolerance latency
Immediate masking High safety	Power/area penalty Voter critical
Immediate masking Long life and high safety	Power/area penalty Switch-voter critical



Contents

- Handling Faults
- Interwoven Redundant Logic
- Combinatorial Modeling
- TMR & its application
- Replication & Voting
- Alternating Logic
- Fail-safe Systems

Inherent Fault Masking in Logic Circuits



$0 \rightarrow 1$ fault in *b* is critical

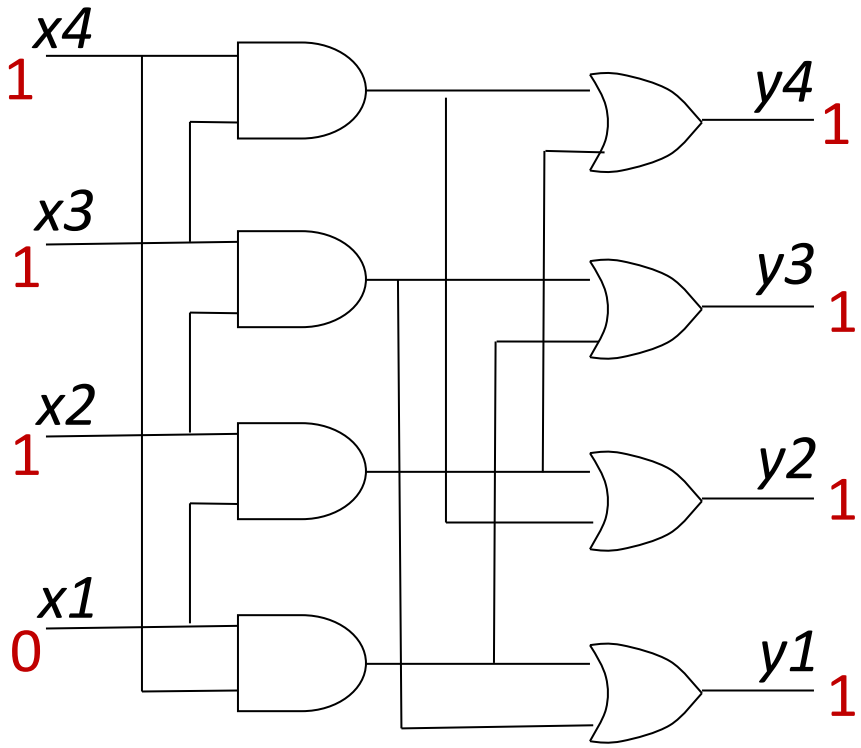
$0 \rightarrow 1$ fault in *c* or *d* is not critical (it is masked)

$1 \rightarrow 0$ fault in *a* or *h* is not critical (it is masked)

Even nonredundant circuits have some masking capability

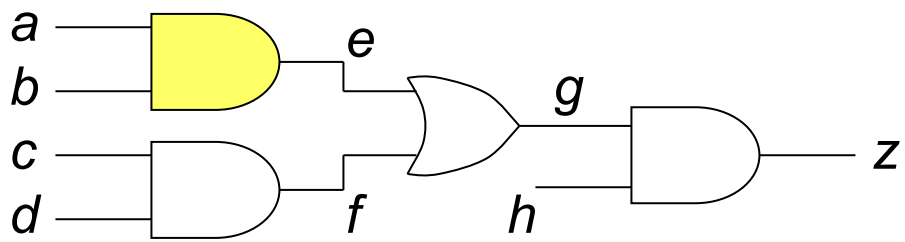
Is there a way to exploit the inherent masking capabilities of logic gates to achieve fault tolerance?

AND-OR Logic Mask

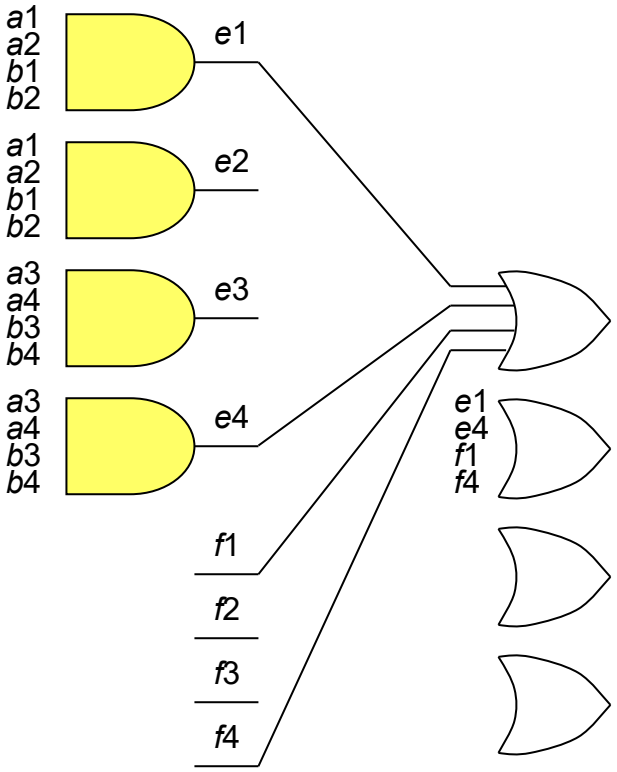


- Single error can be masked
- NAND-NAND Logic

Interwoven Redundant Logic



Let $x1$, $x2$, $x3$, and $x4$ be 4 copies of the signal x



$1 \rightarrow 0$ change is critical for AND, subcritical for OR

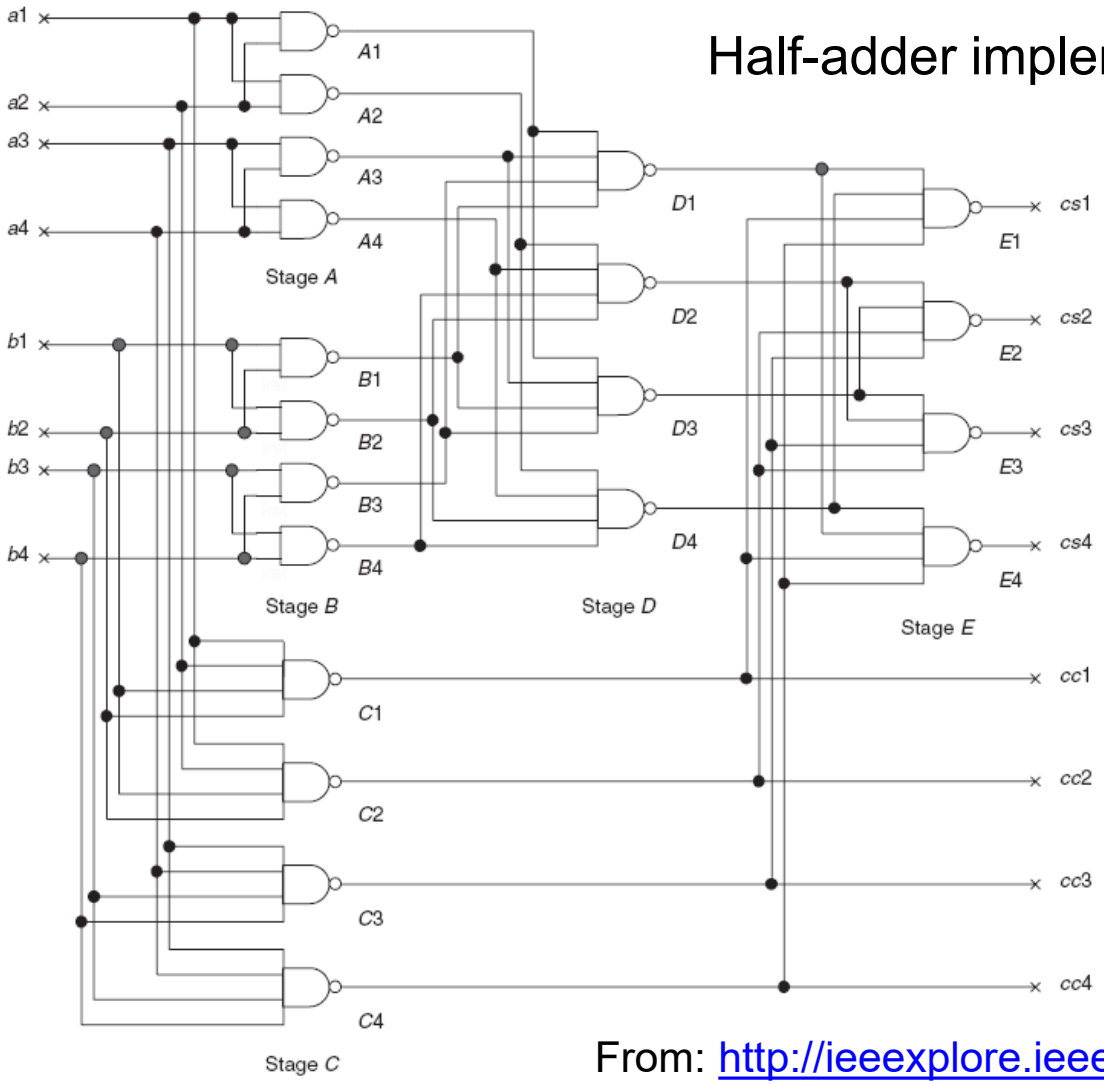
$0 \rightarrow 1$ change is critical for OR, subcritical for AND

Alternating layers of ANDs and ORs can mask each other's critical faults

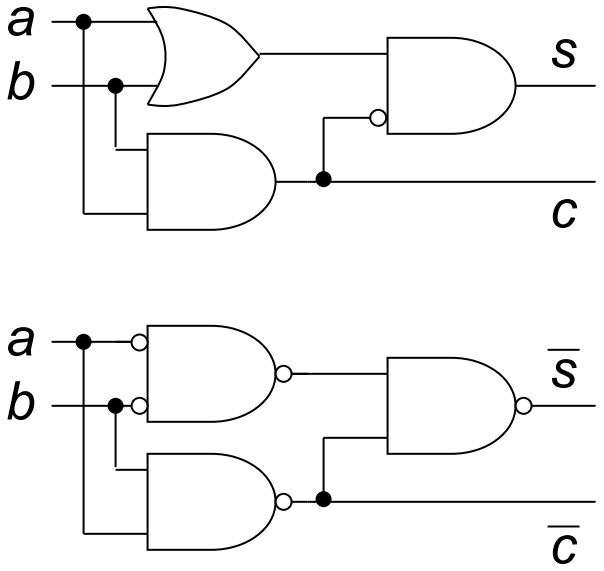
To mask h critical faults:
Number of gates multiplied by $(h + 1)^2$
Gate inputs multiplied by $h + 1$

For $h = 1$, the scheme is known as Quadded logic

Interwoven Logic for Half Adder



Half-adder implemented in quadded logic



IEEE D&T
July-Aug. 2005
pp. 328-339

From: http://ieeexplore.ieee.org/iel5/54/32070/01492293.pdf

Contents

- Handling Faults
- Interwoven Redundant Logic
- Combinatorial Modeling
- TMR & its application
- Replication & Voting
- Alternating Logic
- Fail-safe Systems

Combinatorial Modeling

- System is divided into non-overlapping modules
- Each module is assigned either a probability of working, Pi, or a probability as function of time, Ri(t)
- The goal is to derive the probability, Psys, or function Rsys(t) of correct system operation
- Assumptions:
  - Module failures are independent
  - Once a module has failed, always be assumed to yield incorrect results
  - System is considered failed if don't satisfy minimal set of functioning modules

Reliability of series systems



- Reliability Ri = Prob that component i works
- Reliability Rs = Prob that system s works

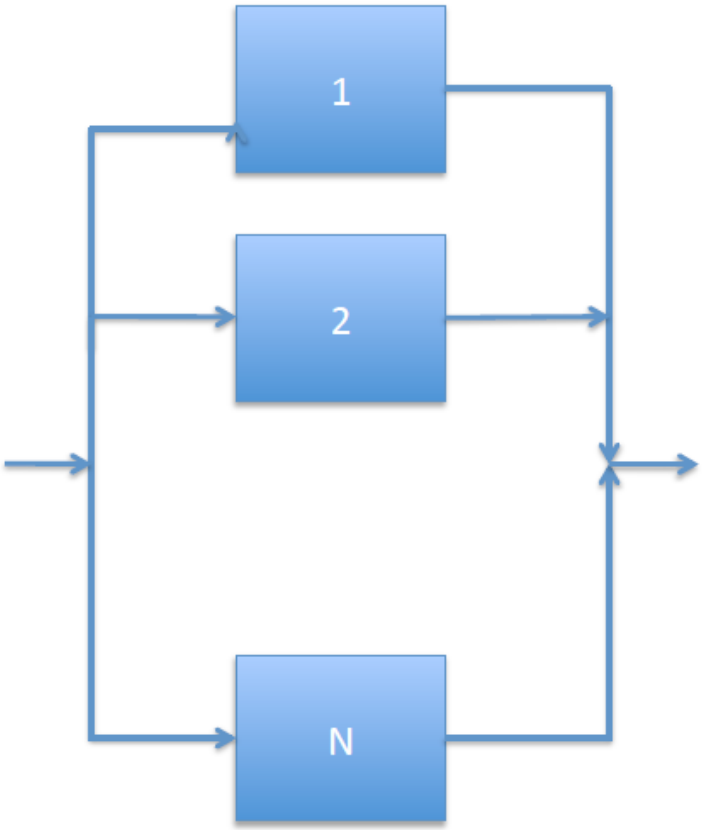
Assume that components fail independently

Component i fails with probability pi

Ri = Prob that comp i works = 1 – pi

Rs = Prob that system works= R1R2R3…Rn = $\prod$Ri

Reliability of parallel systems



- Assume that components fail independently
- Probability that system works =

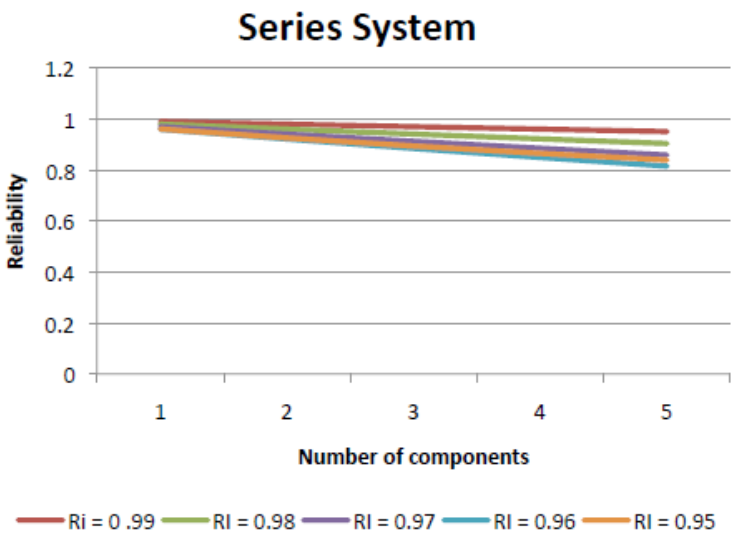
  Rp = 1 – Prob of all components failing

  = 1 – P1P2P3 … Pn

  = 1 - $\prod_0^n(1 - Ri)$

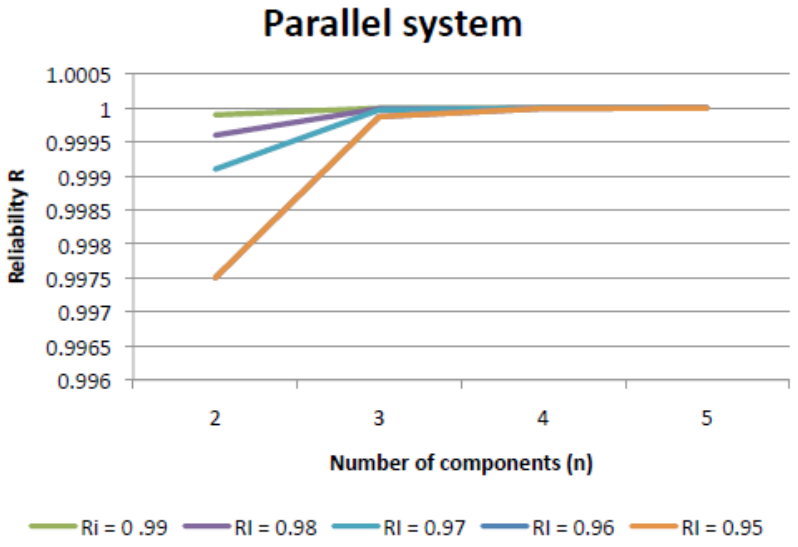
Series-Parallel Effects

- Series system Reliability



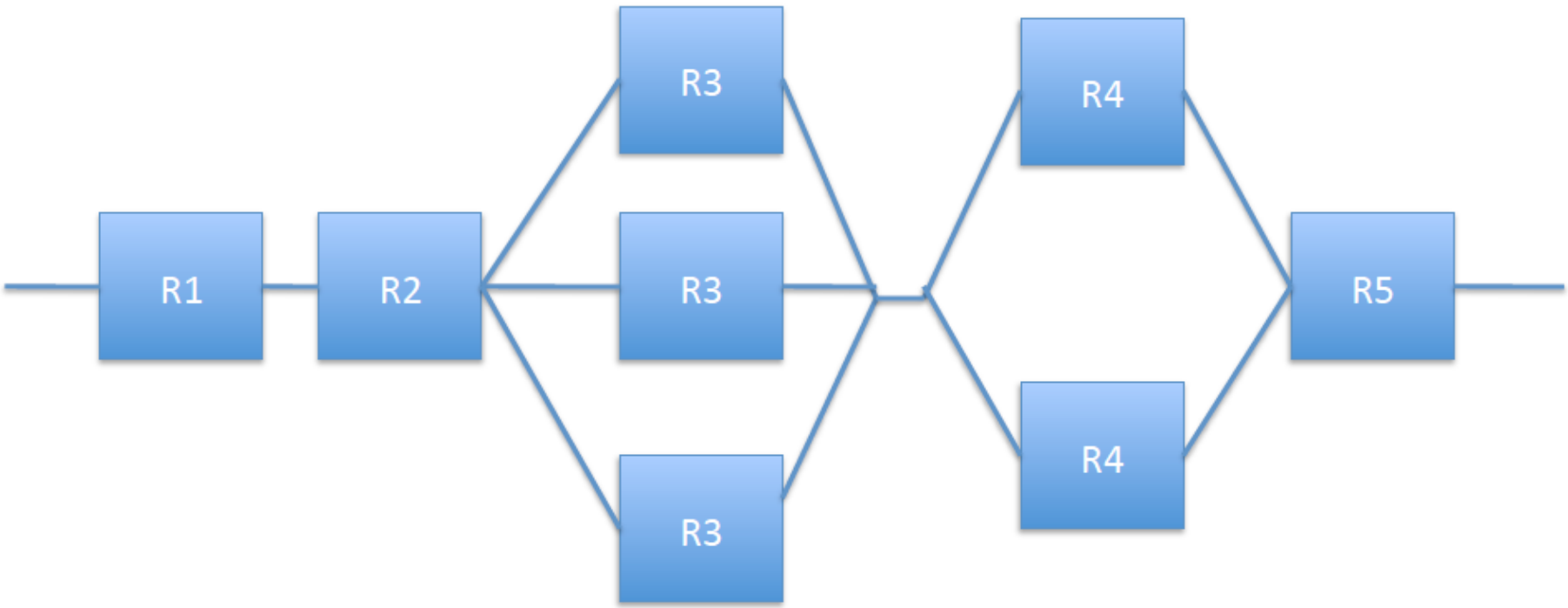
Reliability decreases with N steadily

- Parallel system Reliability



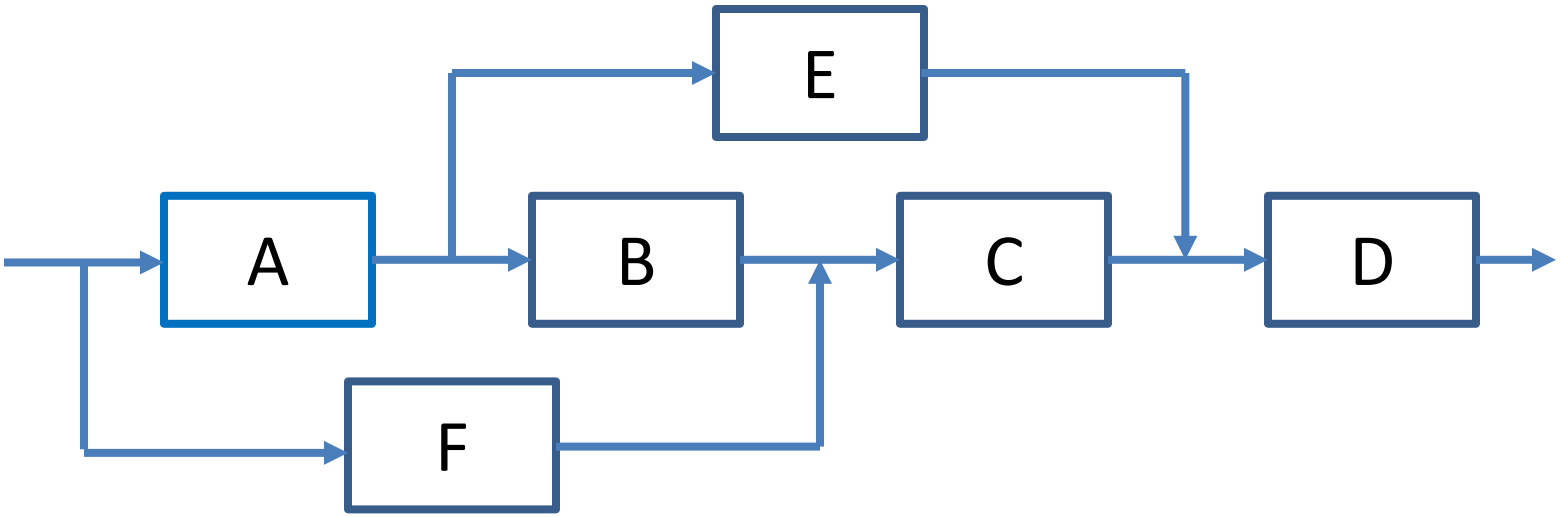
Reliability increases sharply with N

ex. Calculate the reliability of the following system



Non-Series Parallel System

- Neither a series nor parallel system
- How to evaluate the Reliability?



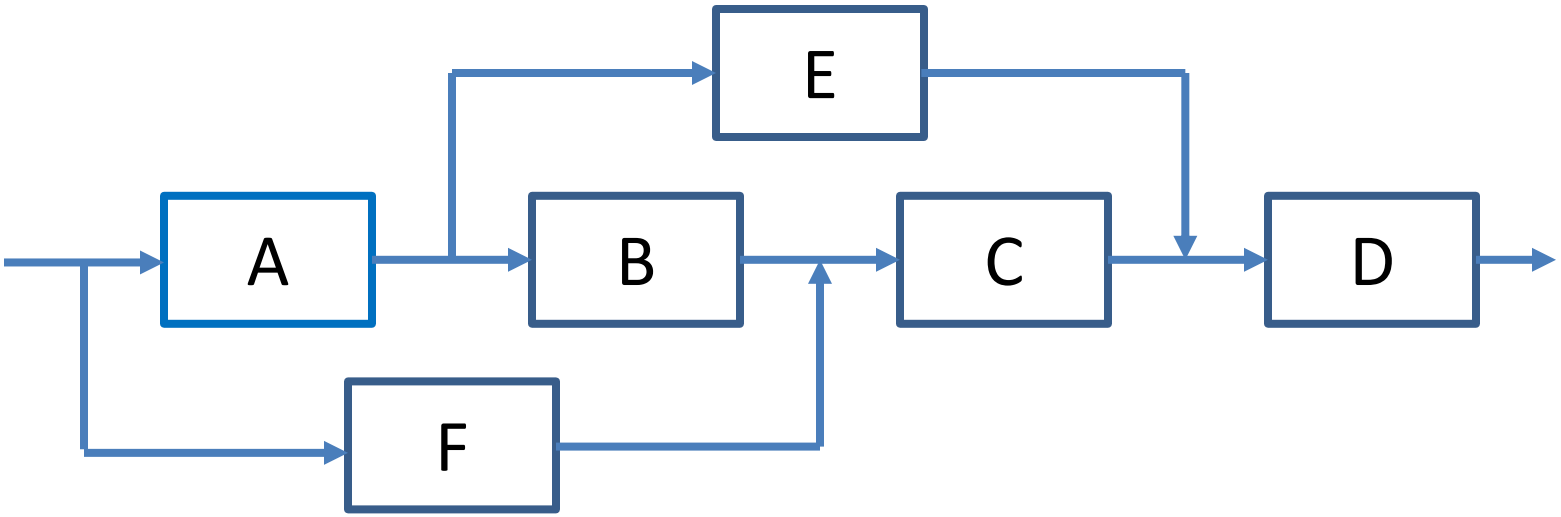
Non-Series Parallel System - 1

- Pick a module ‘m’ in the system

P(sys works) = P(sys works | m works) P(m works)

+ P(sys works | m fails) P(m fails)

Rsys = P(sys works | m)Rm + P(sys works | m’)(1 – Rm)

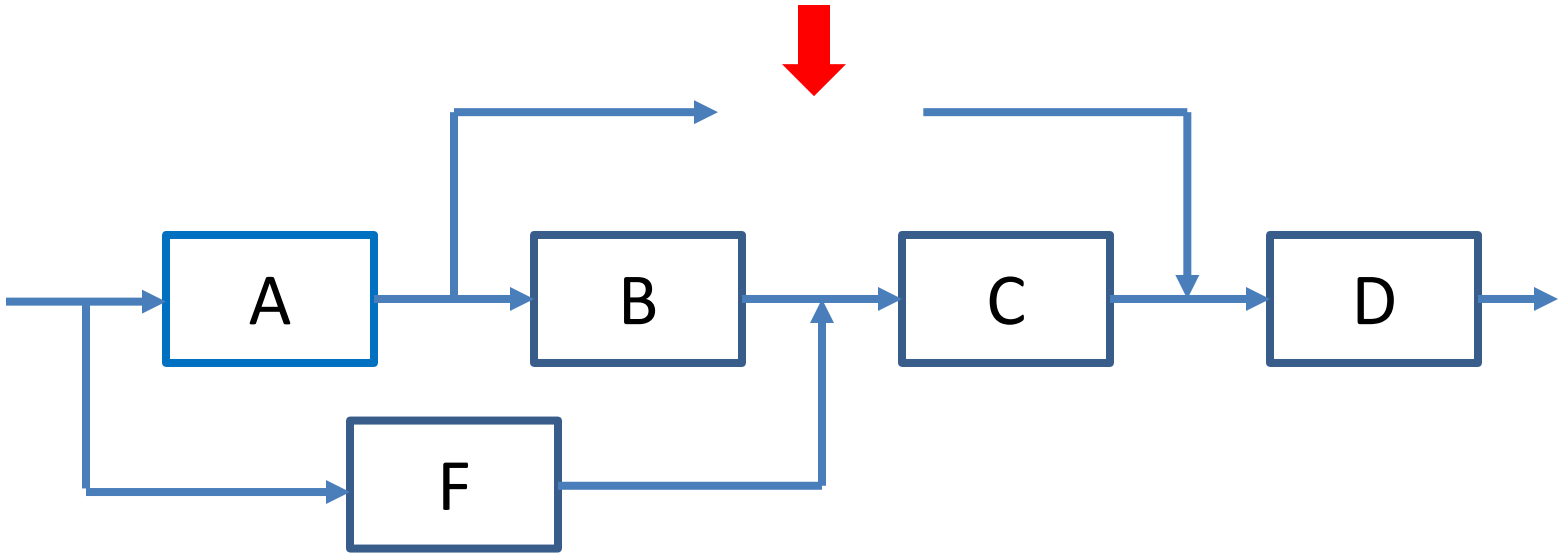


Non-Series Parallel System - 2

- Pick ‘m’ to be module E. Consider ‘E fails’;

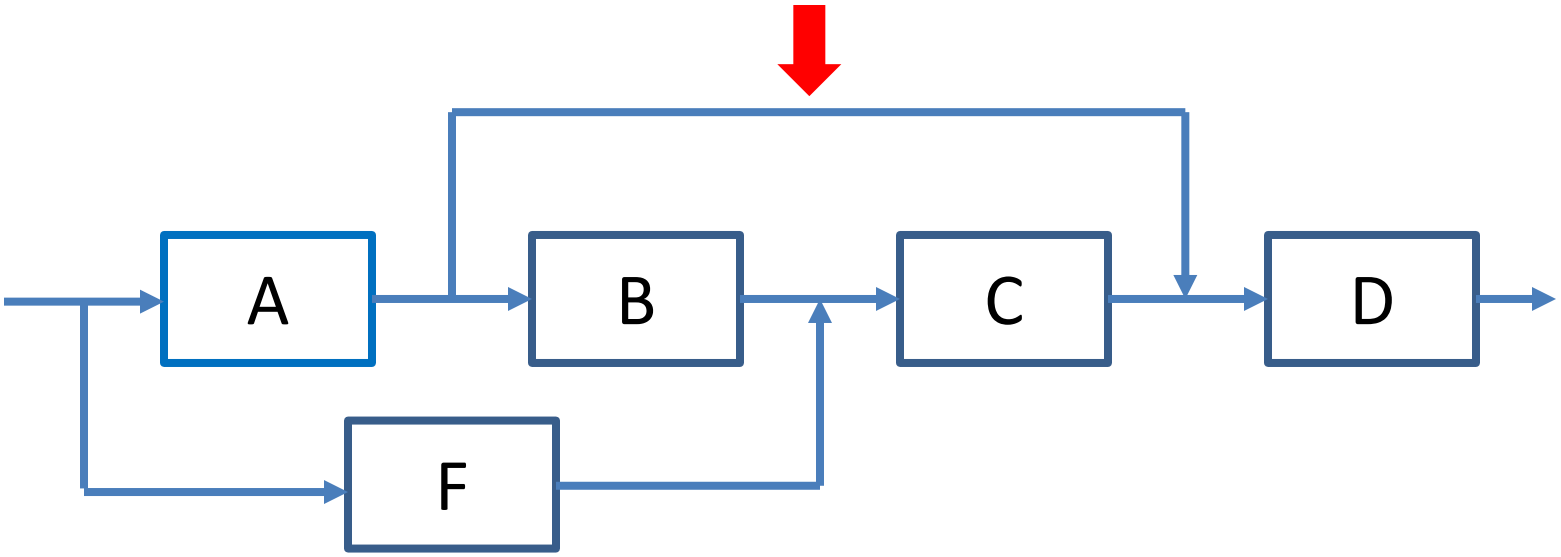
$$P(\text{sys works} \mid E') = [1 - (1 - R_A R_B)(1 - R_F)]R_C R_D$$

‘module E fails’ = ‘path E opened’



Non-Series Parallel System - 3

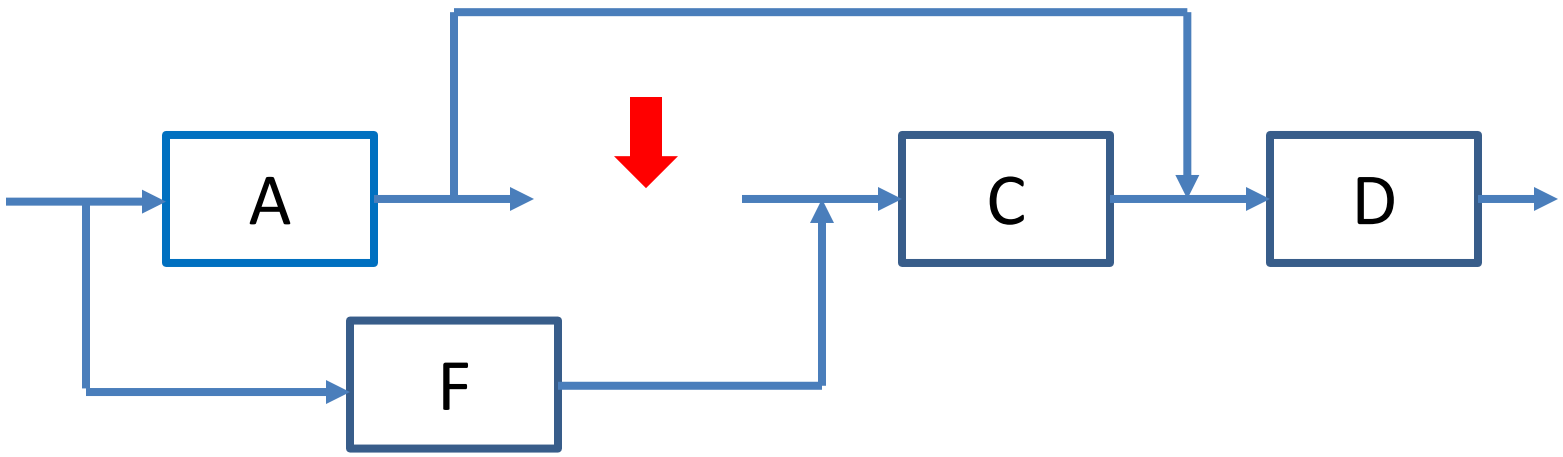
- Next, consider ‘E works’ = ‘path E shorted’
- Still unsolved, do again wt a different module



Non-Series Parallel System - 4

- Next, consider 'B fails' = 'path B opened'

$P(\text{sys works} \mid E \text{ and } B') = [1-(1-R_A)(1-R_F R_C)]R_D$

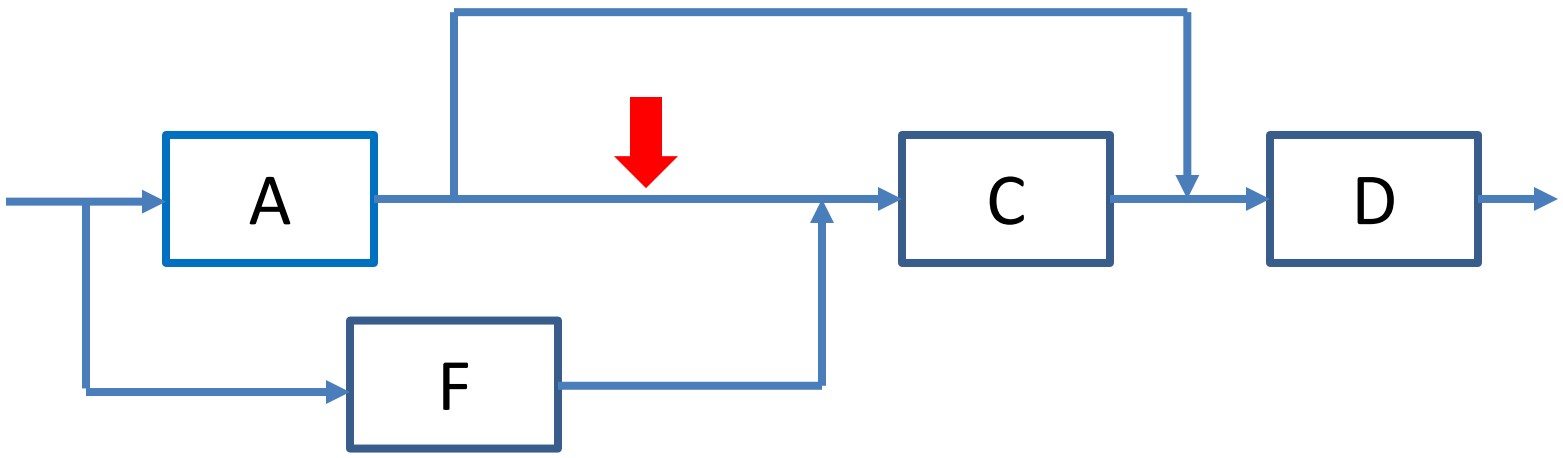


Non-Series Parallel System - 5

- Next, when B works, still the same as before

$$P(\text{sys works} \mid E \text{ and } B) = [1-(1-R_A)(1-R_F R_C)]R_D$$

'module B works' = 'path B shorted'



Putting it all together - 1

P(sys works|E') = $[1 - (1 - R_A R_B)(1 - R_F)]R_C R_D$

P(sys works|E and B') = $[1-(1-R_A)(1-R_F R_C)]R_D$

P(sys works|E and B) = $[1-(1-R_A)(1-R_F R_C)]R_D$

P(sys works|E) = $[1 - (1 - R_A)(1 - R_F R_C)]R_D[R_B + R_B']$

$= [1 - (1 - R_A)(1 - R_F R_C)]R_D$

P(sys works) =

P(sys works|E)R_E + P(sys works|E')(1- R_E)

Putting it all together - 2

Rsys = P(sys works) =

$$[1 - (1 - R_A R_B)(1 - R_F)]R_C R_D (1 - R_E)$$

$$+ [1 - (1 - R_A)(1 - R_F R_C)]R_D R_E$$

$$= [R_E R_D + R_C R_D - R_E R_C R_D][R_A + R_F R_C - R_A R_F R_C]$$

Let $\mathbf{R_A = R_B = R_C = R_D = R_E = R_F = R;}$

Then,

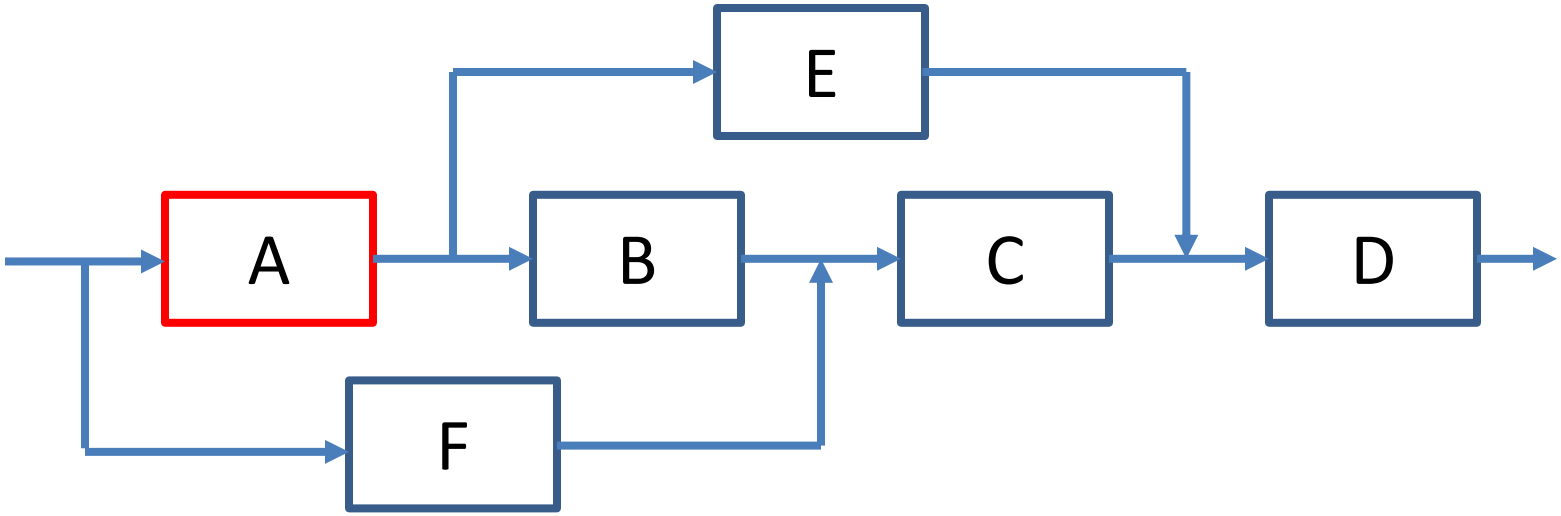
$$\mathbf{Rsys = R^6 - 3R^5 + R^4 + 2R^3}$$

Some tips for non-series-parallel

- Simpler if picked **module A**

- Choosing the initial module is crucial
- Heuristics:
  - Pick modules that are on as many paths as possible
  - Think about whether a single module prevents the system from becoming a serial/parallel system

Non-Series Parallel System (again)

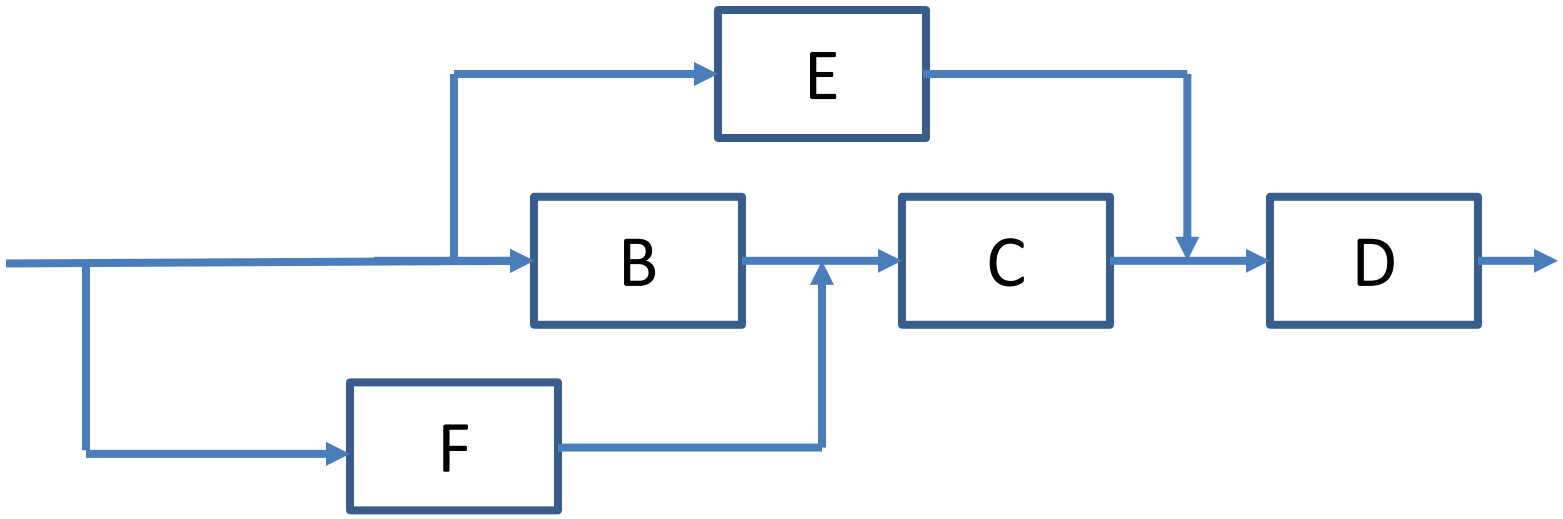


Choose A as the keystone component

$$Rsys = P(sys\ works|A)R_A + P(sys\ works|A')(1-R_A)$$

R:Reliabilty; P:Possibility

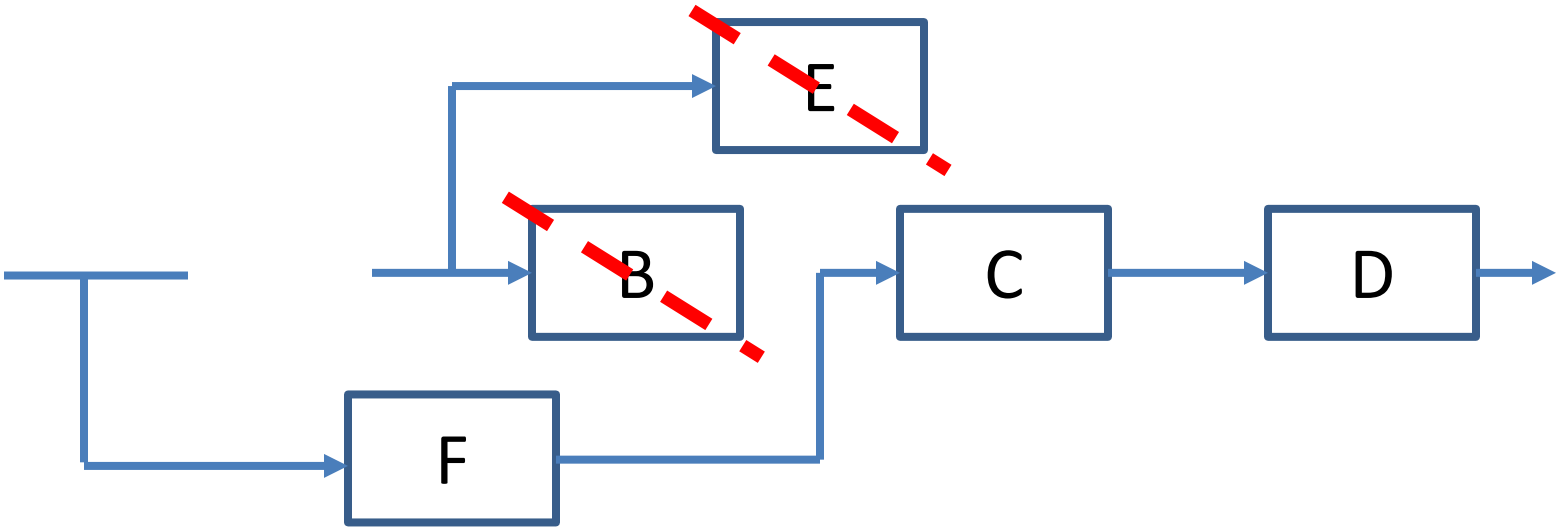
Non-Series Parallel System (again-1)



If A works: $\rightarrow$ *serial/parallel system*

$P(\text{sys works}|A) = (1 - (1 - R_E)(1 - (1 - (1 - R_B)(1 - R_F))\, R_C))\, R_D$

Non-Series Parallel System (again-2)



If A fails: $\rightarrow$ *serial/parallel system*

$P(\text{sys works} \mid A') = R_F R_C R_D$

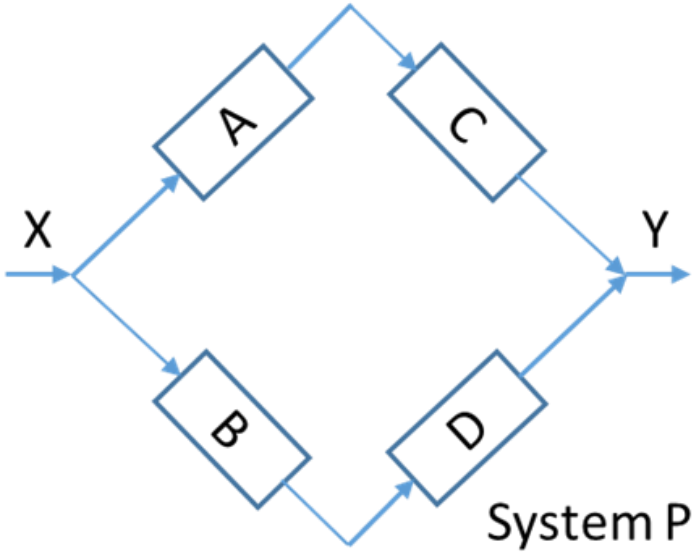
Put it all again

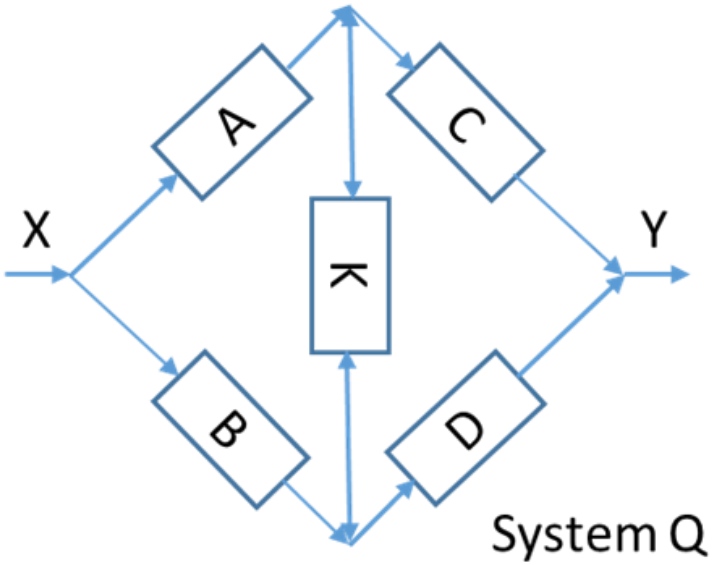
$$
\begin{aligned}
\text{Rsys} &= P(\text{sys works} \mid A)\, R_A + P(\text{sys works} \mid A')\,(1-R_A) \\
&= (1 - (1 - R_E)(1 - (1 - (1 - R_B)(1 - R_F))\, R_C))\, R_D\, R_A \\
&\quad + R_C\, R_D\, R_F\, (1 - R_A) \\
&= R_A\, R_B\, R_C\, R_D + R_A\, R_B\, R_C\, R_F - R_A\, R_B\, R_C\, R_D\, R_F + R_A\, R_D\, R_E \\
&\quad - R_A\, R_B\, R_C\, R_D\, R_E - R_A\, R_C\, R_D\, R_E\, R_F + R_A\, R_B\, R_C\, R_D\, R_E\, R_F \\
&\quad + R_C\, R_D\, R_F - R_A\, R_C\, R_D\, R_F
\end{aligned}
$$

Let $R_A = R_B = R_C = R_D = R_E = R_F = R$, then

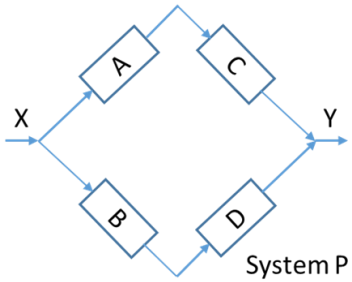
$$\text{Rsys} = R^6 - 3R^5 + R^4 + 2R^3$$

In-class Exercise: Non SP system

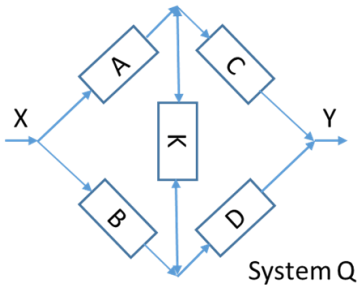




Inclass Exercise: Non SP system



System P



System Q

Ans

System P: Rp = 1-(1-RaRc)(1-RbRd) = 1-(1-R^2)^2 = $2R^2$-R^4

System Q: Rp = P(Sys works|K)Rk + P(Sys works|K')(1-Rk)

P(Sys works|K) = [1-(1-Ra)(1-Rb)][1-(1-Rc)(1-Rd)]

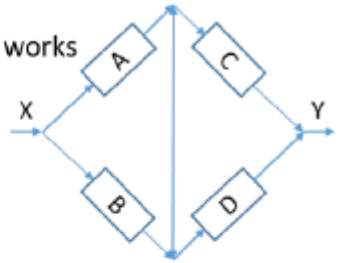
= R^4-$4R^3$+$4R^2$

P(Sys works|K') = 1-(1-RaRc)(1-RbRd) = Rp

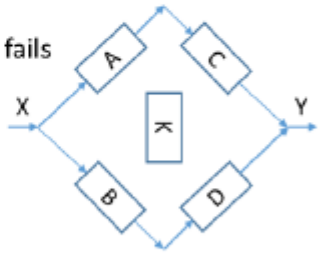
Then, Rq = (R^4-$4R^3$+$4R^2$)Rk + Rp(1-Rk)

= $2R^5$-$5R^4$+$2R^3$+$2R^2$

System Q | K works



System Q | K fails



If Rk is perfect (Rk=1): Rq = [(R^4-$4R^3$+$4R^2$)Rk + Rp(1-Rk)|Rk=1] = R^4-$4R^3$+$4R^2$

Hence: Rq – Rp = (R^4-$4R^3$+$4R^2$) – ($2R^2$-R^4) = $2R^2$(R-1)^2 >= 0

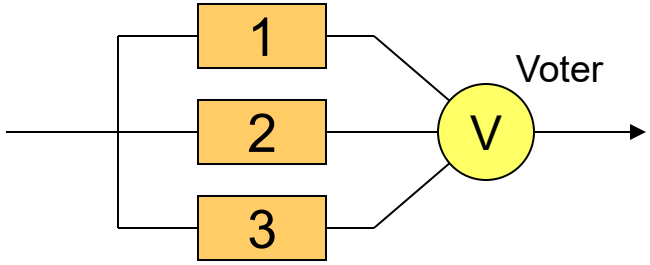
System Q is better reliable than System P

Contents

- Handling Faults
- Interwoven Redundant Logic
- Combinatorial Modeling
- TMR & its application
- Replication & Voting
- Alternating Logic
- Fail-safe Systems

TMR with Perfect Voter

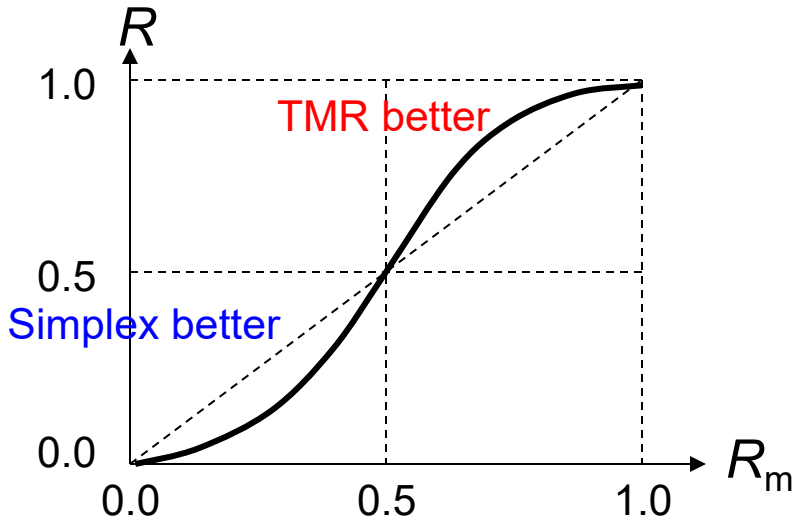
$R = 3R_m^2 - 2R_m^3 \overset{?}{>} R_m$

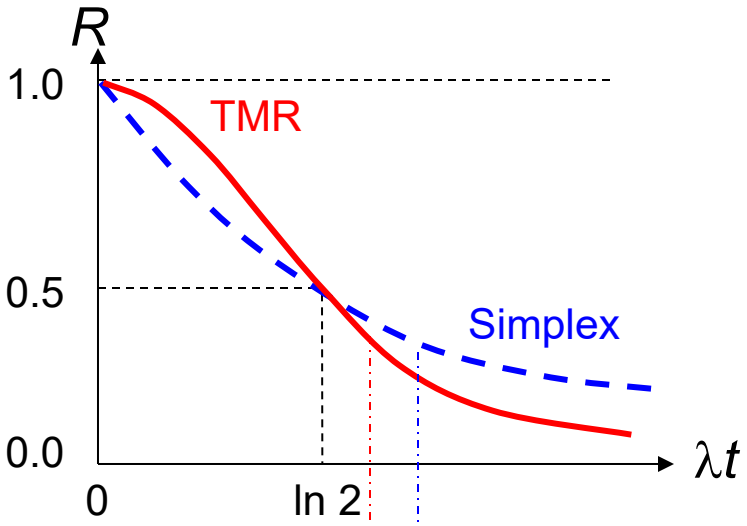


Condition on the module reliability:

$R = R_m [1 + (1 - R_m)(2R_m - 1)]$

$(1 - R_m)(2R_m - 1) > 0 \Rightarrow R_m > 1/2$



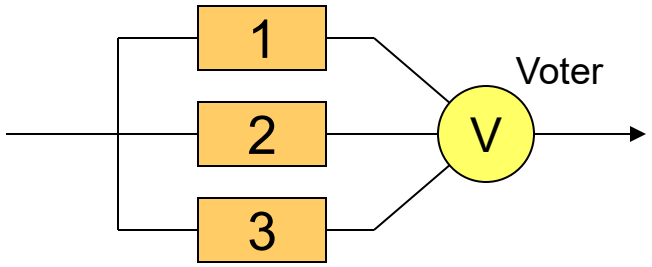


$RIF_{TMR/Simplex} = (1 - R_m)/(1 - R)$
$= 1/[1 - R_m(2R_m - 1)]$

MTTF: TMR $5/(6\lambda)$
Simplex $1/\lambda$

TMR with Imperfect Voter

$R = R_v(3R_m^2 - 2R_m^3) \stackrel{?}{>} R_m$



Condition on the voter reliability

$R_v > 1 / [3R_m - 2R_m^2]$

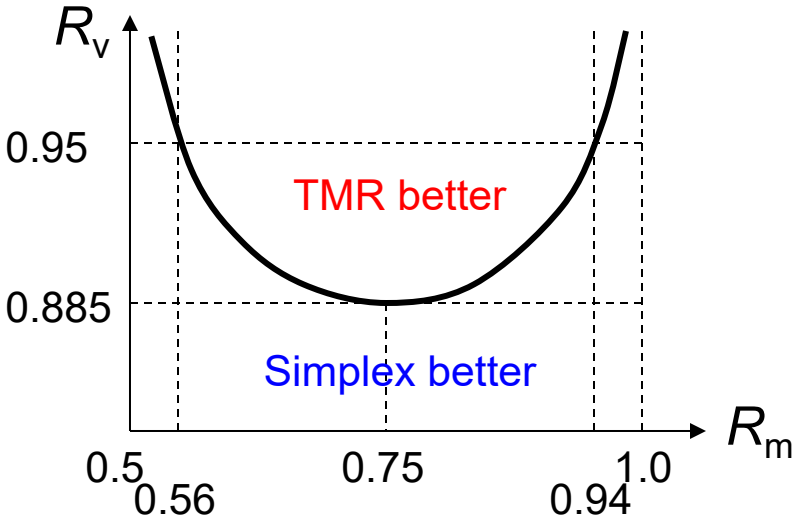
$dR_v^{min} / dR_m = (-3 + 4R_m) / (3R_m - 2R_m^2)^2$

Condition on the module reliability

$$\frac{3 - \sqrt{9 - 8/R_v}}{4} < R_m < \frac{3 + \sqrt{9 - 8/R_v}}{4}$$

Example: $R_v = 0.95$ requires that

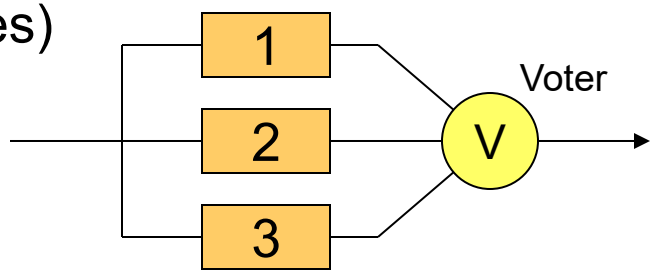
$0.56 < R_m < 0.94$



TMR with Compensating Faults

$R_m = 1 - p_0 - p_1$ (0- and 1-fault probabilities)

$R = (3R_m^2 - 2R_m^3) + 6p_0p_1R_m$



Example: $R_m = 0.998$, $p_0 = p_1 = 0.001$

$R = \underset{\text{Basic TMR}}{\underline{0.999{,}984}} + \underset{\text{Compensation}}{\underline{0.000{,}006}} = 0.999{,}990$

$\text{RIF}_{\text{TMR/Simplex}} = 0.002 / 0.000{,}016 = 125$

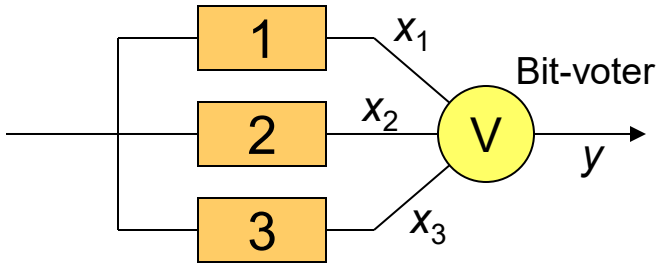
$\text{RIF}_{\text{Compen/TMR}} = 0.000{,}016 / 0.000{,}010 = 1.6$

Contents

- Handling Faults
- Interwoven Redundant Logic
- Combinatorial Modeling
- TMR & its application
- Replication & Voting
- Alternating Logic
- Fail-safe Systems

Implementing a Bit-Voter

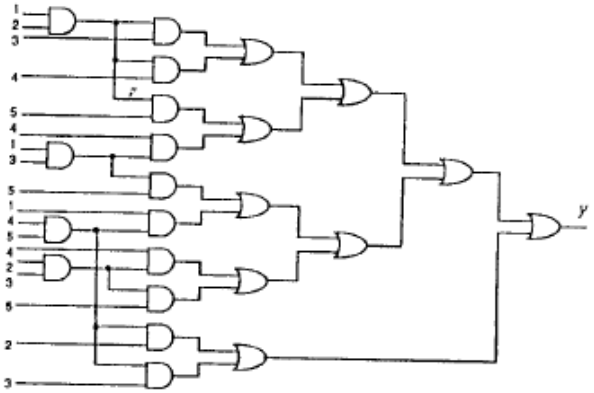
TMR bit-voting: $y = x_1x_2 \vee x_2x_3 \vee x_3x_1$
(carry output of a single-bit full-adder)
What about 5MR, 7MR?



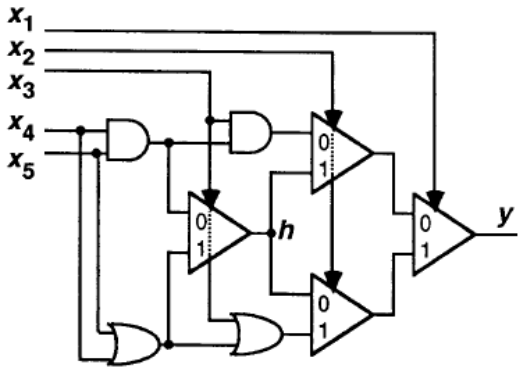
Gate-level design quickly explodes in size

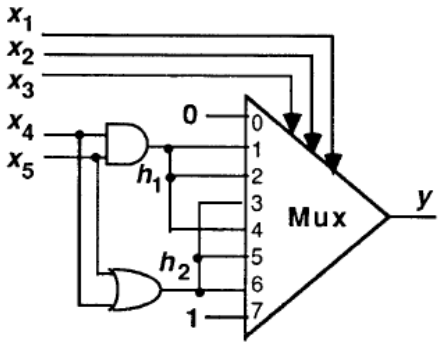
Other designs are also possible
Arithmetic: add the bits, compare to threshold
Mux-based
Selection-based (majority of bit values is their median)

3-out-of-5 voter built of 2-input gates

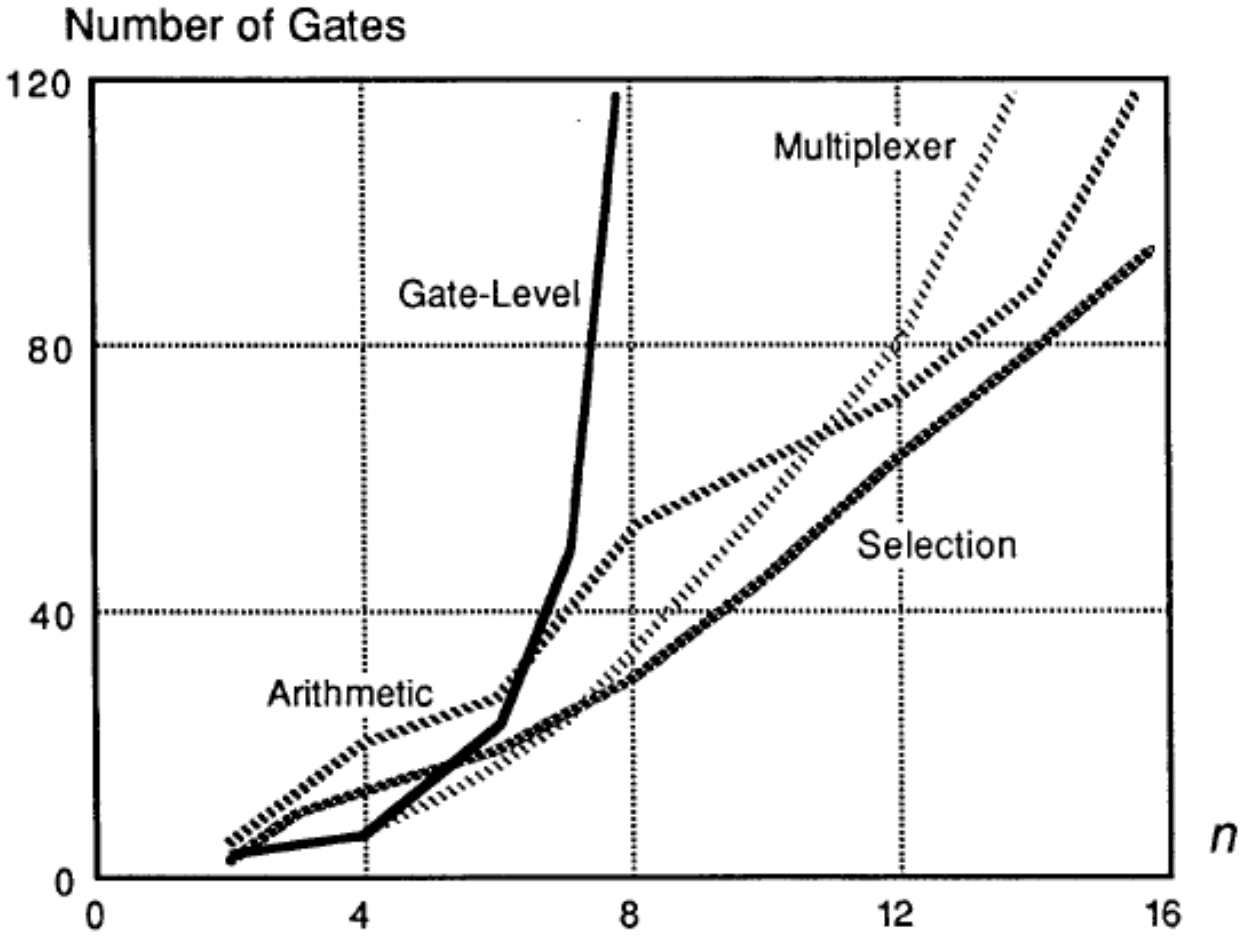


Two mux-based designs for a 3-out-of-5 bit-voter





Complexity of Different Bit-Voter Designs



Cost of majority bit-voters as a function of the number *n* of inputs

Voting at the Word Level

Using bit-by-bit voting may be dangerous

One might think that in this example, any of the module outputs could be correct, so that producing 1 0 at the output isn't all that wrong

$x_1 = 0\ 0$
$x_2 = 1\ 0$
$x_3 = 1\ 1$
$y = 1\ 0$

However, with bit-by-bit voting, the output may be different from all inputs

$x_1 = 0\ 0\ 0$
$x_2 = 1\ 0\ 1$
$x_3 = 1\ 1\ 0$
$y = 1\ 0\ 0$

Design of bit- and word-voting networks discussed in:
Parhami, B., "Voting Networks," *IEEE TR*, Aug. 1991

Bit-Voting and Word-Voting (1)

Scenario1: every module has an error bit at the output

Expected output is (00000)

Module Number	Word (5 bits)					*Value (decimal)*
	Bit 4	*Bit 3*	*Bit 2*	*Bit 1*	*Bit 0*	
#1	0	1	0	0	0	**8**
#2	0	1	0	0	0	**8**
#3	0	0	1	0	0	4
#4	0	0	0	1	0	2
#5	0	0	0	0	1	1
Bit-Voter Result	0	0	0	0	0	**0**
Word-Voter Result	0	1	0	0	0	**8**

maintaining data integrity

Bit-Voting and Word-Voting (2)

Scenario2: some modules have multiple errors

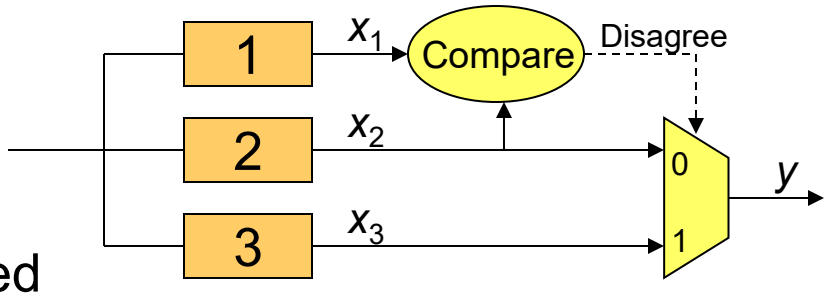
Expected output is (00000)

Module Number	Word (5 bits)					*Value (decimal)*
	Bit 4	*Bit 3*	*Bit 2*	*Bit 1*	*Bit 0*	
#1	0	0	0	0	0	**0**
#2	1	0	1	1	1	23
#3	0	0	0	0	0	**0**
#4	1	1	0	0	1	25
#5	1	1	1	0	1	27
Bit-Voter Result	1	0	0	0	1	**17**
Word-Voter Result	0	0	0	0	0	**0**

maintaining data integrity

Some Simple Voter Designs

If in the case of 3-way disagreement any of the inputs can be chosen, then a simple design is possible



This design can be readily generalized to a larger number of inputs

One can perform pseudo voting that yields the median of 3 analog signals (Dennis, N.G., *Microelectronics and Reliability*, Aug. 1974)

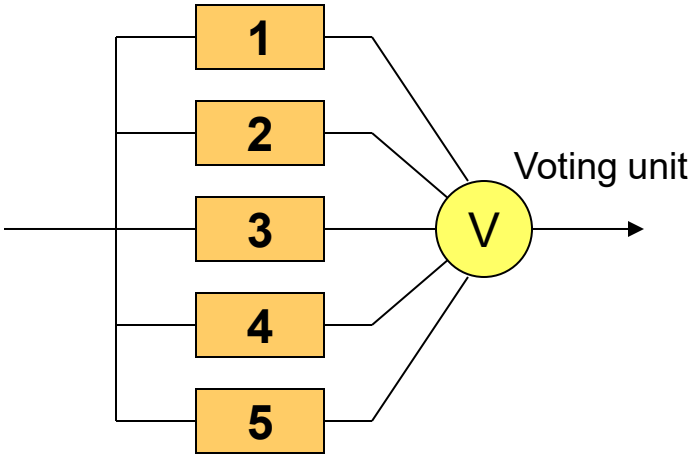
Median and mean voting are also possible with digital signals

N-Modular Redundancy (NMR)

Triple-modular redundancy (TMR) can be generalized to *N* units

N-modular redundancy (NMR) uses *N* modules along with a voter, with *N* usually being odd

Example: 5MR
Operates correctly as long as
3 of the 5 modules are healthy



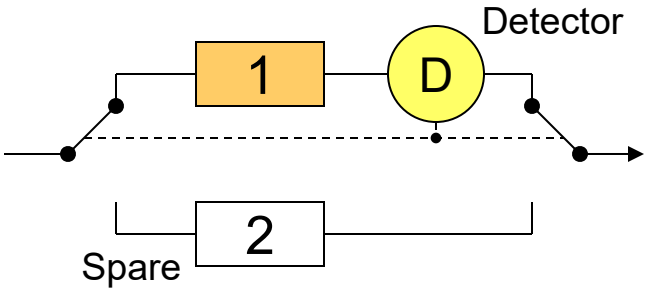
Voter complexity rises rapidly
with increasing *N*

Even values of *N* are also feasible

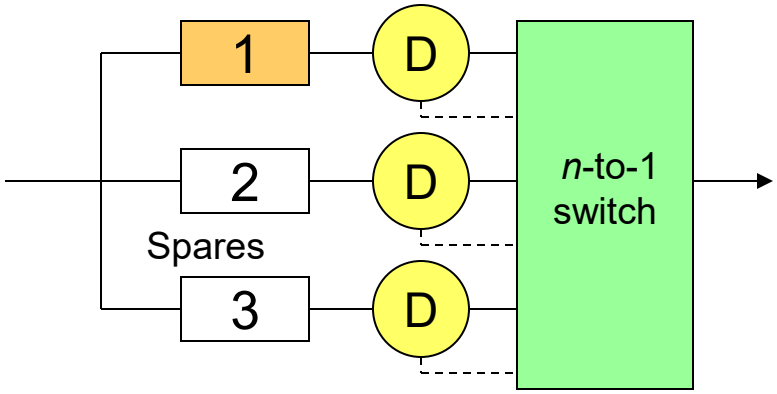
Example: 4MR, with 3-out-of-4 voting
Voter masks single faults; can be designed to detect double faults

Switch for Standby Redundancy

Standby redundancy requires an *n*-to-1 switch to select the output of the currently active module



The detectors use various info to deduce fault conditions
-- Error coding
-- Reasonableness checks
-- Watchdog timer



Once a fault has been detected, the switch reconfigures the system by flagging the faulty unit and activating next spare in sequence

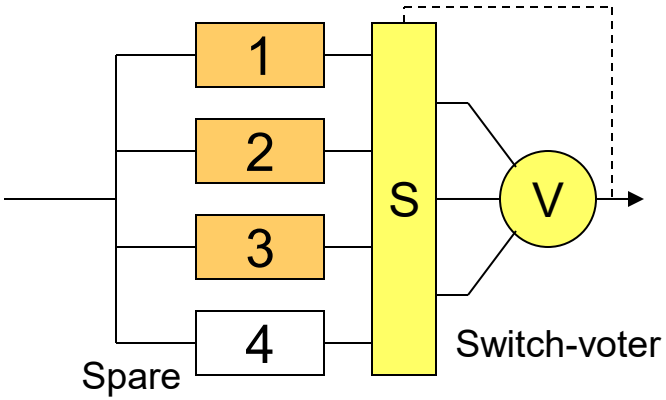
If we use an *n*-to-2 switch and compare the two selected outputs, the configuration is known as “pair-and-spares”

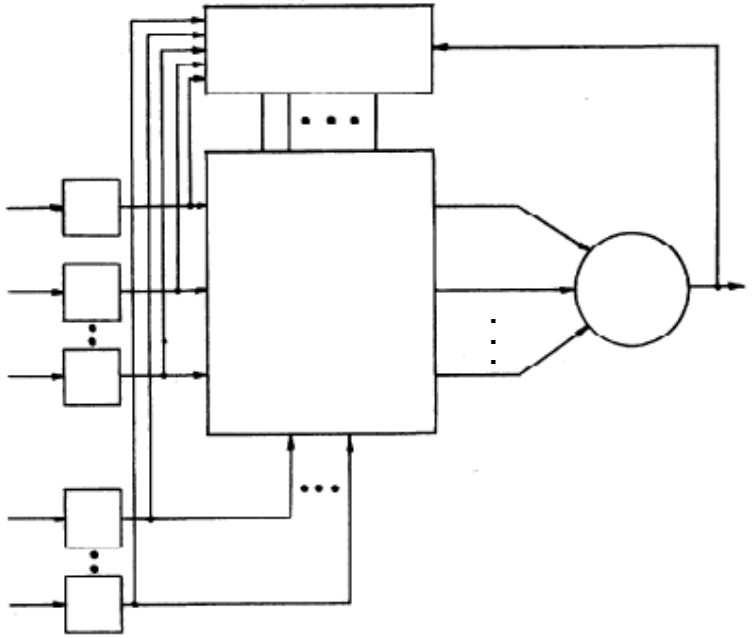
Switch for Hybrid Redundancy

Hybrid redundancy with *n* active and *s* spare modules requires an (*n* + *s*)-to-*n* switch to select the outputs of the active modules

Self-purging redundancy is a variant of hybrid redundancy in which all modules are active at the outset, but they are purged as they disagree with the majority output

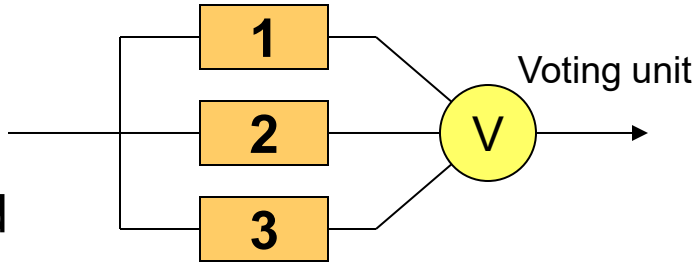
Voter in self-purging redundancy is a threshold voter that considers the inputs with weights of 1 (active) or 0 (purged)





Variations and Design Issues

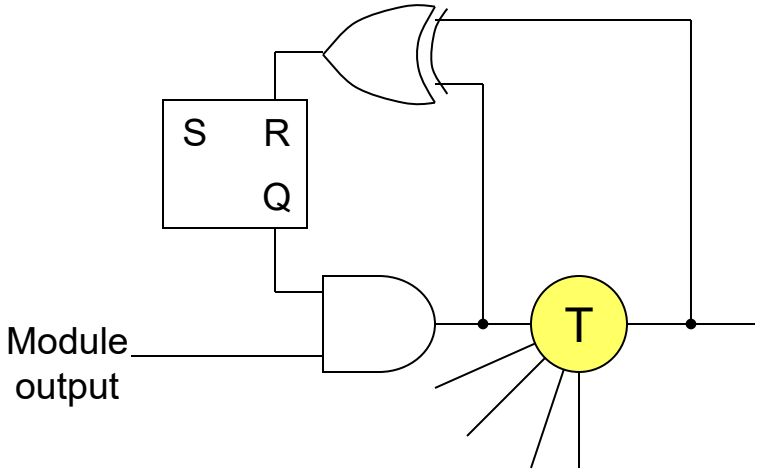
NMR/simplex: Voting unit is replaced with a unit that can also detects disagreements



When a faulty unit is detected, that unit and one other unit are removed from service

This makes all votes unambiguous and also improves systems lifetime

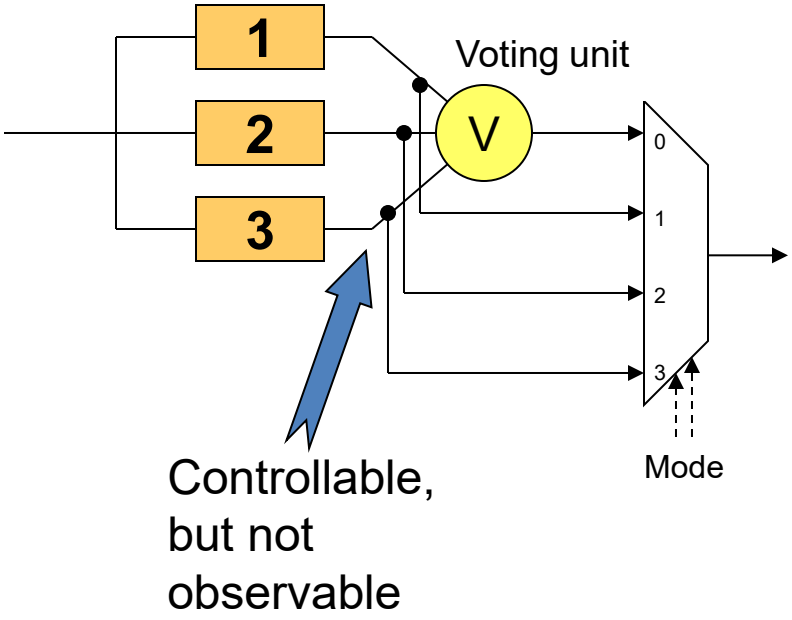
Self-purging redundancy: Modules purged when they disagree with the output and the threshold of the voting unit is adjusted accordingly (purged modules produce 0 outputs)



Variations and Complications

Static redundancy makes fault testing more challenging

For static redundancy to be effective, we must ensure that initially all redundant components are fault-free



Time Redundancy

Retry upon a detected fault: particularly useful for transient faults

Recomputation not useful with permanent faults

Can make recomputation work by slightly changing the operands, but this is not always applicable

Compute $a \times (2b)$ instead of $(2a) \times b$

Compute $b + a$ or $-(-a - b)$ instead of $a + b$

Applications of *n*MR and Hybrid Redundancy

The Space Shuttle:

Uses 5-way redundancy in hardware
- Originally, 3 operational units + 2 spares (one warm, one cold)
- More recently, 4 operational + 1 spare

Also, uses 2 independently developed software systems (Design diversity)



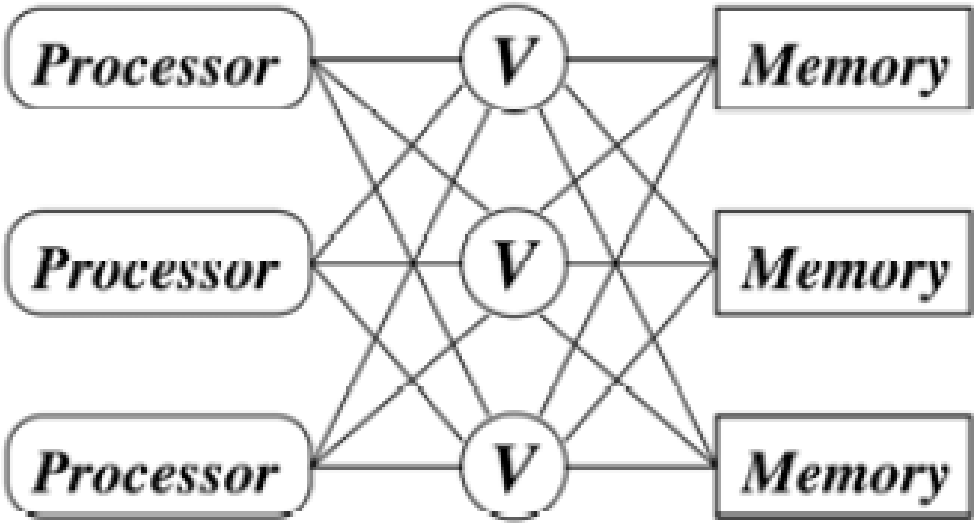


Japanese Shinkansen “Bullet” Train

Triple-duplex system (6-fold redundancy)

Check yourself

Q. List the conditions under which it will fail?



Hardware Redundancy Recap

Data path methods:

Replication in space (costly)
- Duplicate and compare
- Triplicate and vote
- Pair-and-spare
- NMR/hybrid

Replication in time (slow?)
- Recompute and compare
- Recompute and vote
- Alternating logic
- Recompute after shift
- Recompute after swap
- Replicate operand segments

Mixed space-time replication

Monitoring (imperfect coverage)
- Watchdog timer
- Activity monitor

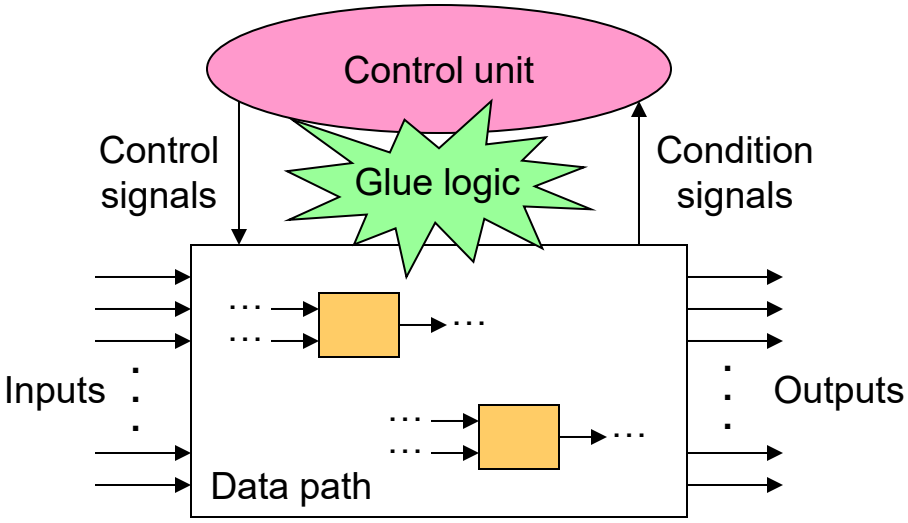
Low-redundancy coding
- Parity prediction
- Residue checking
- Self-checking design

Control unit methods:

Coding of control signals
Control-flow watchdog
Self-checking design

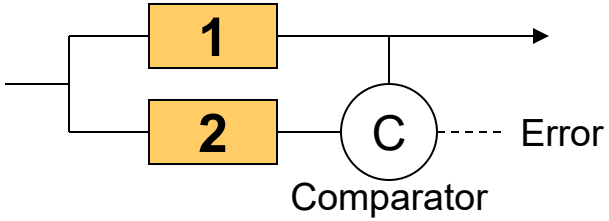
Glue logic methods:

Self-checking design

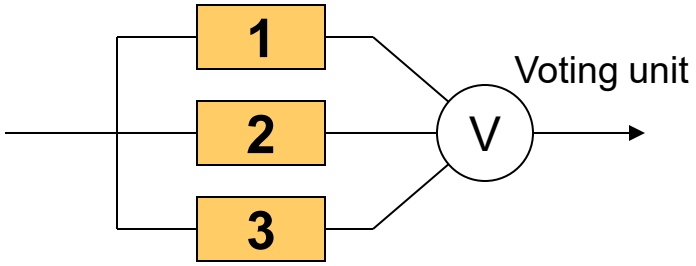


Replication in Space

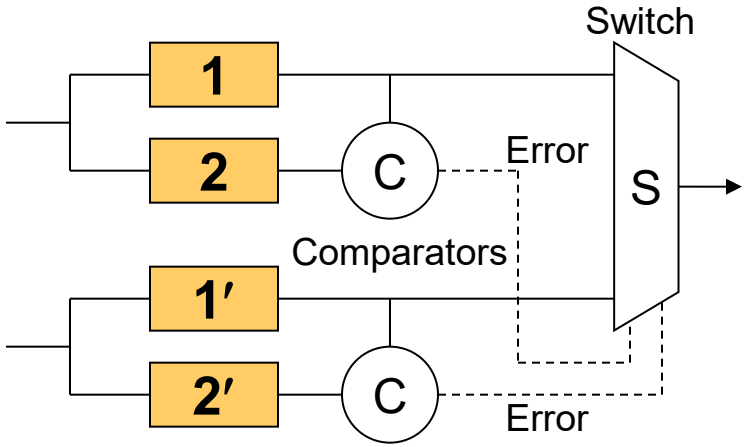
The following schemes have already been discussed in connection with fault masking



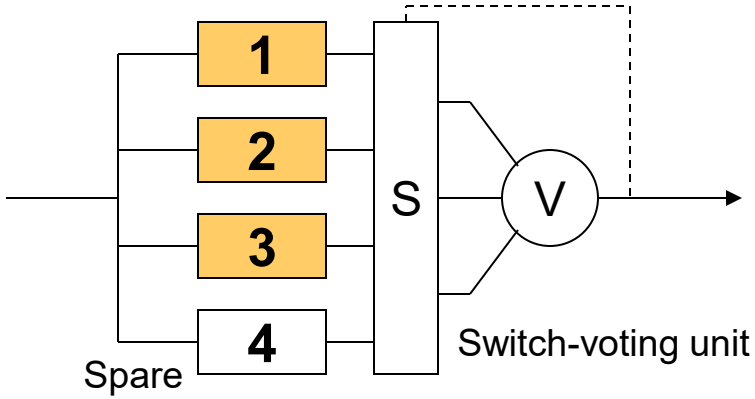
Duplicate and compare



Triplicate and vote



Pair-and-spare

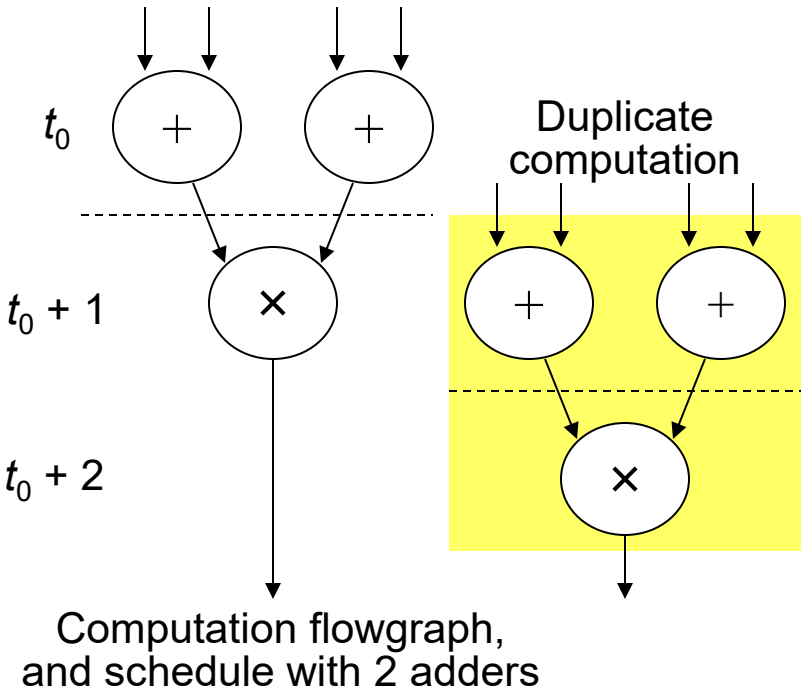


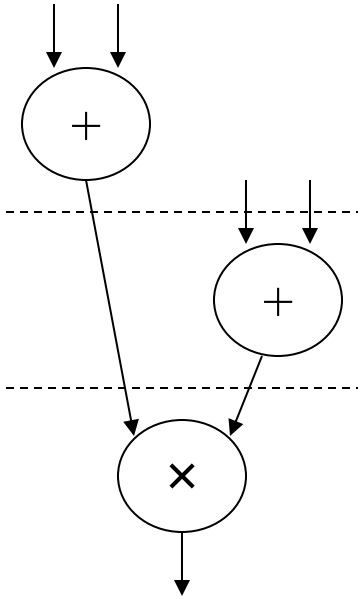
NMR/Hybrid

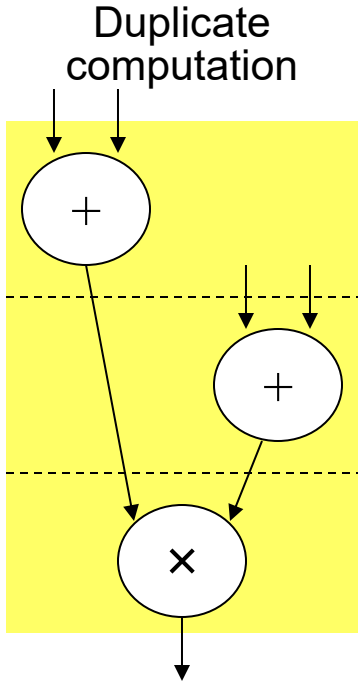
Replication in Time

Can be slow, but in many control applications, extra time is available

Interleaving of the primary and duplicate computations saves time

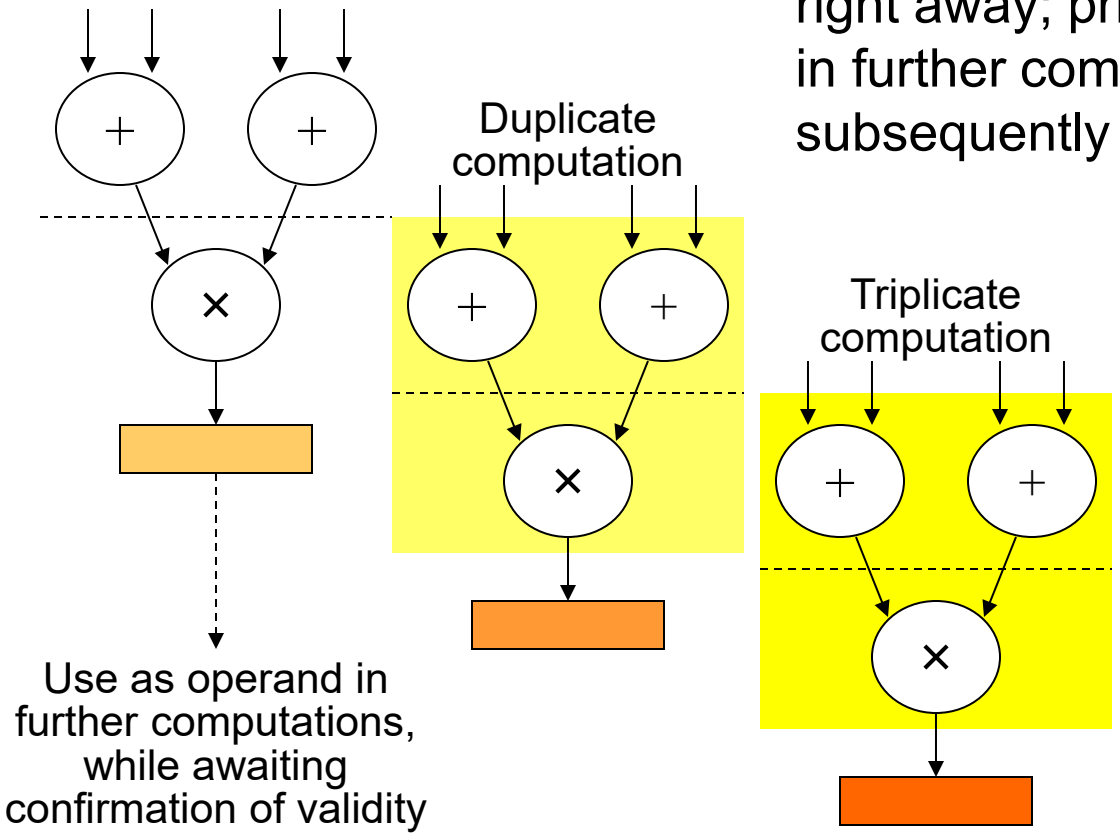






Recompute and Compare / Vote

Repeat computation and store the results for comparison or voting



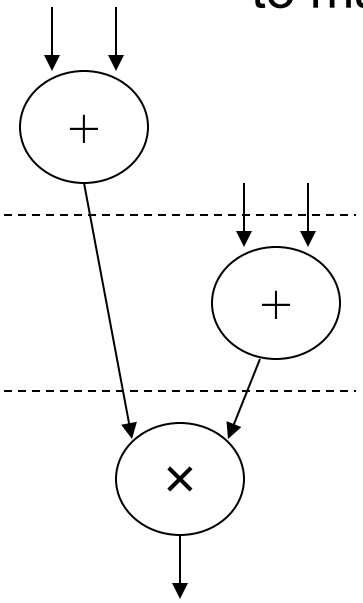
Comparison or voting need not be done right away; primary result may be used in further computations, with the result subsequently validated, if appropriate

On a simultaneous multithreading architecture, multiple instruction streams may be interspersed

Some Cray machines take advantage of extensive hardware resources to execute instructions twice

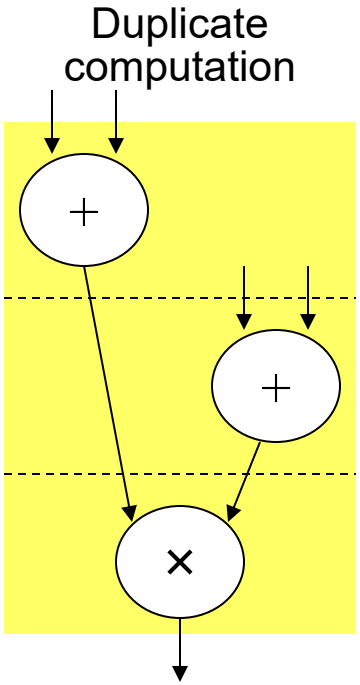
Mixed Space/Time Replication

Instead of duplicating the computation with no hardware change (slow) or duplicating the entire hardware (costly), we can add some hardware to make the interleaved recomputations more efficient

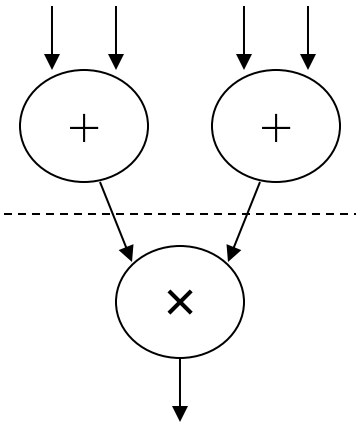


Original computation (T = 3)

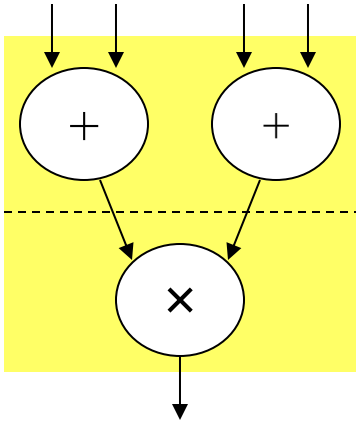
Recomputation with same hardware resources (T = 5, excluding compare time)



Consider the effect of including a second adder



Recomputation with the inclusion of an extra adder (T = 3, excluding compare time)



Error Detection by Duplicated Instructions (EDDI)

- The redundant code is compared before Store

Original Code

```
add r1, r2, r3       // r1 = r2 + r3
xor r4, r1, r5       // r4 = r1 XOR r5
store r4, 0($r6)     // Mem[$r6] = r4
```

Code with EDDI-like Redundancy

```
add r1, r2, r3       // r1 = r2 + r3
add r11, r12, r13    // r11 = r12 + r13
xor r4, r1, r5       // r4 = r1 XOR r5
xor r14, r11, r15    // r14 = r11 XOR r15
bne r4, r14, error   // if r4 !=r14, goto error
store r4, 0($r6)     // Mem[$r6] = r4
```

Contents

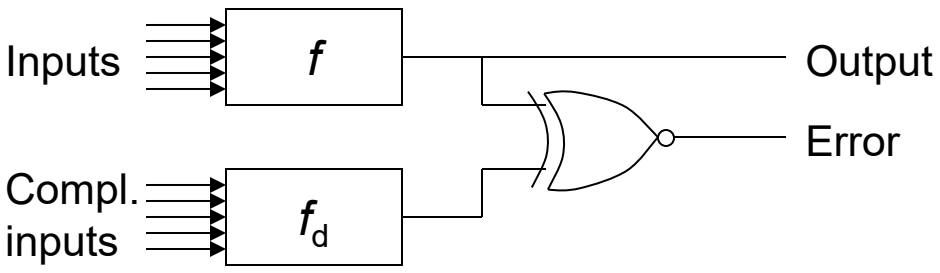
- Handling Faults
- Interwoven Redundant Logic
- Combinatorial Modeling
- TMR & its application
- Replication & Voting
- Alternating Logic
- Fail-safe Systems

Alternating Logic: Basic Ideas

Transmission of data over unreliable wires or buses
- Send data; store at receiving end
- Send bitwise complement of data
- Compare the two versions
- Detects wires s-a-0 or s-a-1, as well as many transients

The *dual* of a Boolean function $f(x_1, x_2, \ldots, x_n)$ is another function $f_d(x_1, x_2, \ldots, x_n)$ such that $f_d(x_1', x_2', \ldots, x_n') = f'(x_1, x_2, \ldots, x_n)$

Fact: Obtain the dual of f by exchanging AND and OR operators in its logical expression. For example, the dual of $f = ab \vee c$ is $f_d = (a \vee b)c$

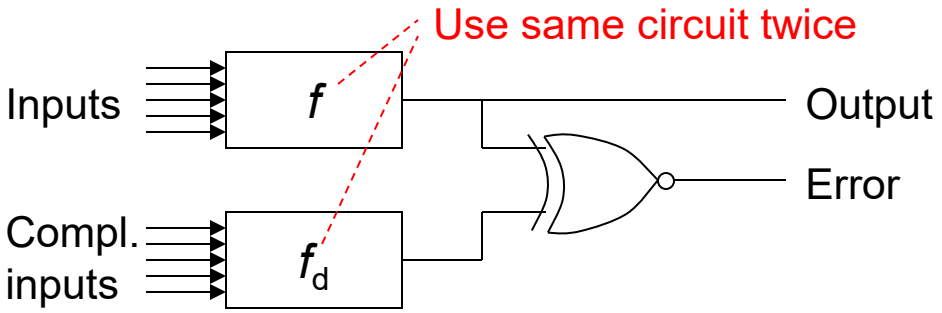


Advantages of this approach compared with duplication include a smaller probability of common errors

Alternating Logic: Self-Dual Functions

A function f is self-dual if $f(x_1, x_2, \ldots, x_n) = f_d(x_1, x_2, \ldots, x_n)$

For example, both the sum $a \oplus b \oplus c$ and carry $ab \vee bc \vee ca$ outputs of a full-adder are self-dual functions



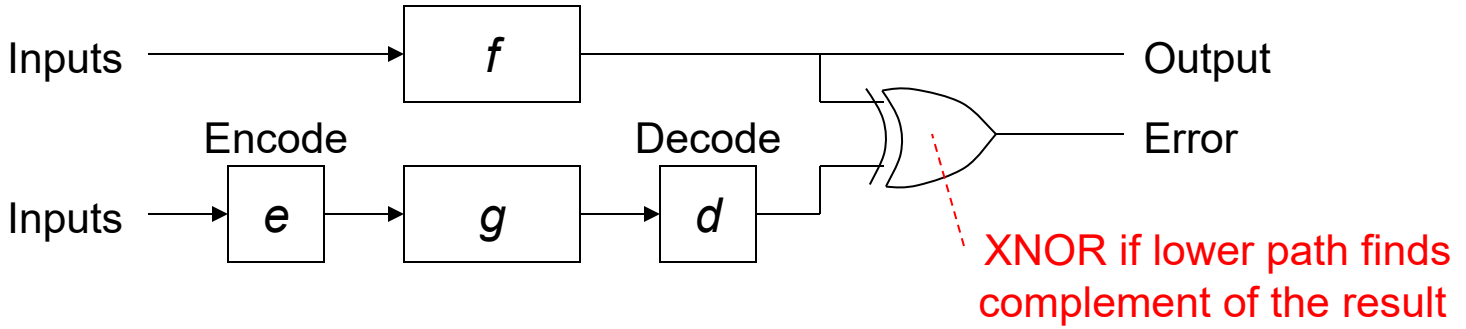
With a self-dual function f, the functions f and f_d in the diagram above can be computed by using the same circuit twice (time redundancy)

Many functions of practical interest are self-dual

Examples (proofs left as exercise)
A k-bit binary adder, with $2k + 1$ inputs and $k + 1$ outputs, is self-dual
So are 1's-complement and 2's-complement versions of such an adder

Recomputing with Transformed Operands

Alternating logic is a special case of the following general scheme, with its encoding and decoding functions being bitwise complementation



Recompute after shift

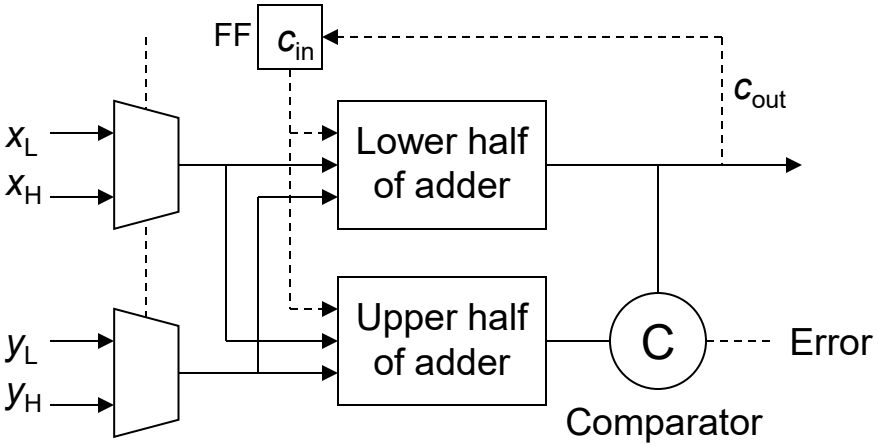
When *f* is binary addition, we can use shifts for encoding and decoding
Shifting causes the adder circuits to be exercised differently each time
Originally proposed for ALUs with bit-slice organization

Recompute after swap

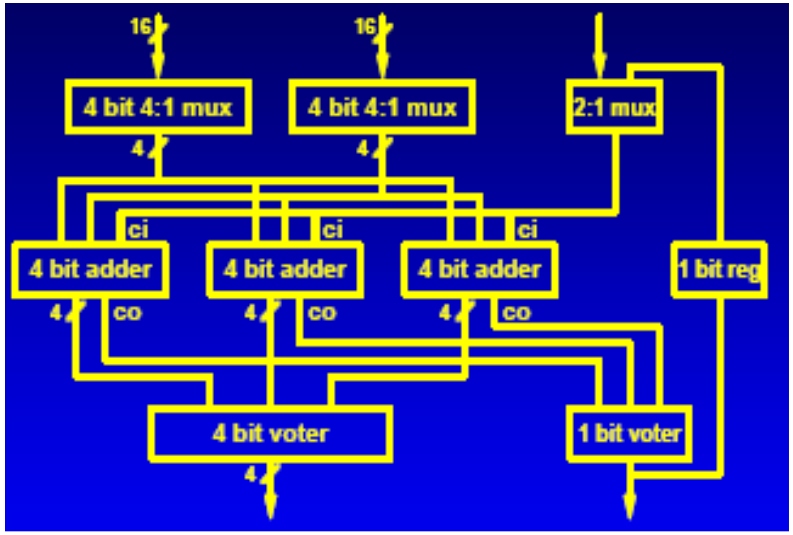
When *f* is binary addition, we can use swaps for encoding and decoding
- Swap the two operands; e.g., compute *b* + *a* instead of *a* + *b*
- Swap upper and lower halves of the two operands (modified adder)

Time-Redundant, Segmented Addition

Instead of using a *k*-bit adder twice for error detection or 3 times for error correction, one can segment the operands into 2 or 3 parts and similarly segment the adder; perform replicated addition on operand segments and use comparison/voting to detect/correct error



Sum computed in two cycles: The lower half in cycle 1, and the upper half in cycle 2



Townsend, Abraham, and Swartzlander, 2003

Various other segmentation schemes have been suggested

Example: 16-bit adder with 4-way segmentation and voting

Contents

- Handling Faults
- Interwoven Redundant Logic
- Combinatorial Modeling
- TMR & its application
- Replication & Voting
- Alternating Logic
- Fail-safe Systems

Fail-Safe Systems

Fail-safe: Produces one of a predetermined set of safe outputs when it fails as a result of “undesirable events” that it cannot tolerate

Fail-safe traffic light: Will remain stuck on red

Fail-safe gas range/furnace pilot: Cooling off of the pilot assembly due to the flame going out will shut off the gas intake valve

A fail-safe digital system must have at least two binary output lines, together representing the normal outputs and the safe failure condition

Reason: If we have a single output line, then even if one value (say, 0) is inherently safe, the output stuck at the other value would be unsafe

Two-rail encoding is a possible choice: 0: 01, 1: 10, F: 00, 11, or both

Totally fail-safe: Only safe erroneous outputs are produced, provided another failure does not occur before detection of the current one

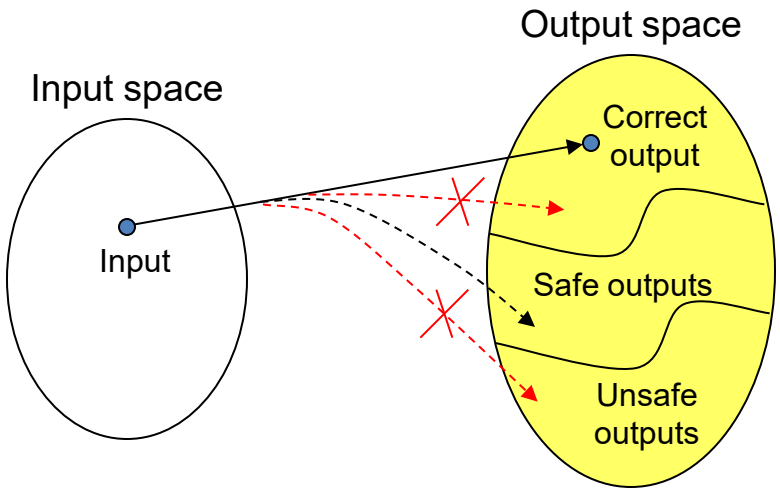
Ultimate fail-safe: Only safe erroneous output is produced, forever

Fail-Safe System Specification

Amusement park train safety system

Signal s_B when asserted indicates that the train is at beginning of its track (can move forward, but should not be allowed to go back)

Signal s_E when asserted indicates that the train is at end of its track (can go back, but should not move forward)



Is the specification above consistent and complete?

No, because it does not say what happens if $s_B = s_E = 1$; this would not occur under normal conditions, but because such sensors are often designed to fail in the safe mode, the combination is not impossible

Why is this a problem, though? (Train simply cannot be moved at all)

Completeness will prevent potential implementation or safety problems

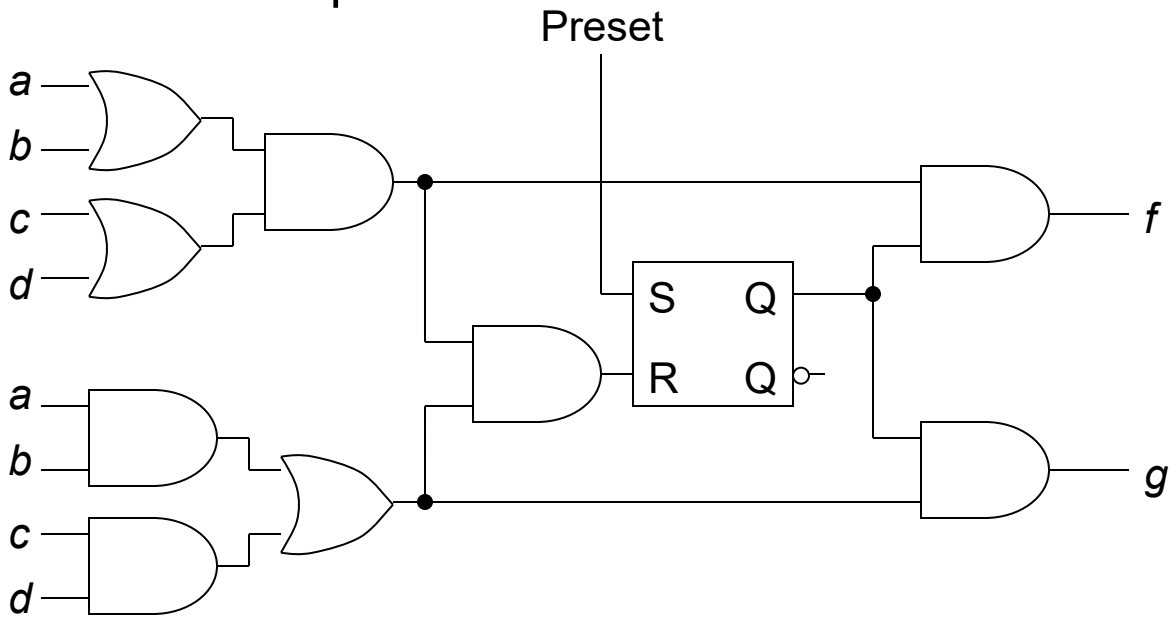
Fail-Safe 2-out-of-4 Code Checker

Input: 4 bits *abcd*, exactly 2 of which must be 1s

Output: *fg* = 01 or 10, if the input is valid
00 safe erroneous output
11 unsafe erroneous output

Codewords

a	*b*	*c*	*d*
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0



Output will become permanently 00 upon the first unsafe condition

Fail-Safe State Machines

Use an error code to encode states

Implement the next-state logic so that the machine is forced to an error state when something goes wrong

Possible design methodology:

Use Berger code for states, avoiding the all 0s state with all-1s check, and vice versa

Implement next-state logic equations in sum-of-products form for the main state bits and in product-of-sums form for the check state bits

The resulting state machine will be fail-safe under unidirectional errors

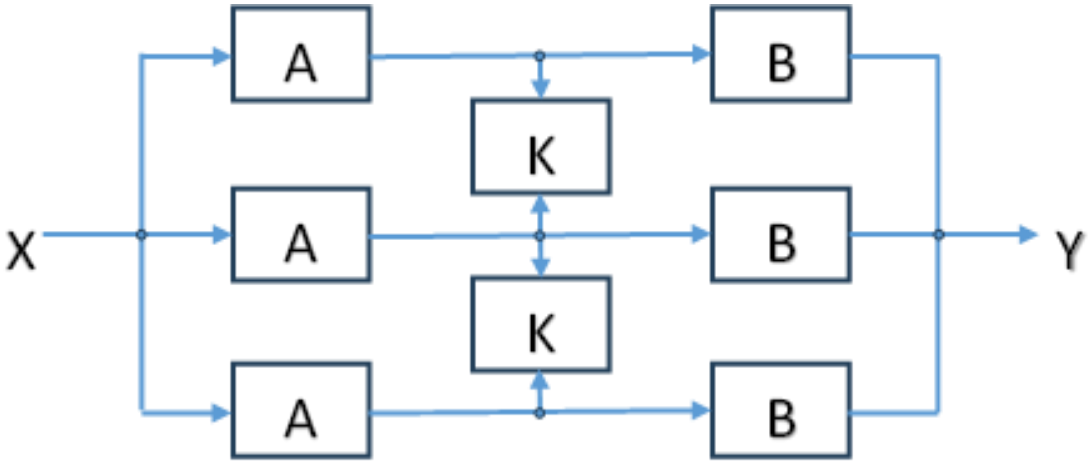
	Input	
State	*x*=0	*x*=1
A	E	B
B	C	D
C	A	D
D	E	D
E	A	D

State	Encoding
A	001 10
B	010 10
C	011 01
D	100 10
E	101 01

Hardware overhead for *n*-state machine consists of O(log log *n*) additional state bits and associated next-state logic, and a Berger code checker connected to state FFs

Home Exercise: Non-SP

- A system operated from X to Y via non serial-parallel path
- Reliabilities of Module (A, B, K) = **(Ra, Rb, Rk)**
- Find the **system reliability**



Midterm Exam

- **On-Site**
- **12 Oct 2026, 13.00-16.00**
- **CP: #17-01, CEDT: #18-15 Eng.Bldg.no.4**
- **All about Fault Tolerant Computing**
- **Open Books, Open Net**